

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM



SREE SASTHA INSTITUTE OF ENGINEERING AND TECHNOLOGY

Team ID : SWTID-2026-1606

Team Size : 4

Team Leader : Nandhini K

Email ID : knandhini0852@gmail.com

Team Member : Sowmiya U

Email ID : Soumisoumiya693@gmail.com

Team Member : Harish Vel A

Email ID : harishvel550@gmail.com

Team Member : T Saranya

Email ID : Saranyathavamani18@gmail.com

Salesforce-Based Multi-Line Insurance Policy and Claims Management System

1. Problem Statement

An insurance provider handling Auto, Property, and Life insurance is currently dependent on a legacy process for policy quotation, issuance, and claim handling. Because information is maintained across separate processes, quotation and claim activities are slower, less consistent, and harder to monitor. Manual claim review also increases turnaround time, operating effort, and the possibility of an inconsistent customer experience.

To address these limitations, the project proposes a unified Salesforce solution that can:

1. **Standardize and simplify policy quotation and issuance across all supported insurance lines.**
2. **Automate claim creation and route claims according to policy category and claim severity.**
3. Provide a centralized 360-degree view of Customer, Policy, and Claim information.

2. Objectives

- **Create a flexible data model that can store line-specific information such as VIN details for Auto policies and square footage for Property policies.**
- **Use Salesforce Screen Flows to guide agents through structured, multi-step quotation processes for different policy types.**
- **Develop an Apex and Lightning Web Component (LWC) based claims dashboard that helps adjusters view and manage assigned claims efficiently.**
- **Automate claim routing, approval, and escalation based on claim value and related policy information using Record-Triggered Flows.**
- Apply validation rules and controlled data entry to improve accuracy and maintain consistent insurance records.

3. Challenges

- **Variable Data Management:** Managing the different data requirements of Auto, Property, and Life insurance within a common object structure using Record Types and Field Sets.
- **Approval Complexity:** Building claim approval logic that can route records dynamically according to policy type, claim amount, and approver responsibility.
- **API Integration Simulation (Apex):** Demonstrating external verification logic through Apex callout simulation and corresponding mock test classes.
- **User Adoption:** Designing simple Screen Flows that reduce unnecessary steps and make the quotation process easier for agents to complete.

4. Solution Approach

The proposed implementation combines Salesforce configuration and custom development. Screen Flows support guided quotation, a flexible custom data model handles multiple policy lines, automation manages claim routing, and Apex with LWC provides a responsive claims dashboard.

1. **Flexible Data Model:** Configure Policy and Claim objects with separate Record Types for Auto, Property, and Life insurance.

2. **Guided Quoting (Declarative):** Build multi-screen Screen Flows that capture common and policy-specific information in a structured sequence.
3. **Claims Logic (Programmatic/Flow):** Use Apex for premium-related logic and Record-Triggered Flows for automated claim routing and updates.
4. **UI Development (LWC):** Create a focused claims dashboard that presents relevant claim information to adjusters.

5. Implementation Milestones

Milestone 1: Core Data Model and Policy Configuration

User Story 1: Configure the Core Insurance Objects

1. Configure the Policy Custom Object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
 - Label: Policy
 - Plural Label: Policies
 - Object Name: Auto Populated
 - Record Name: Policy Number
 - Data type: Auto Number
 - Format: P - {0000}
 - Starting Number: 1
- Under optional features:
 - Allow Reports, Activities and Track field History
- Under Deployment Status:
 - Select the deployed radio button.
- Under Search Status:
 - Check the Allow Search checkbox.
- Click on save.

The screenshot shows the Salesforce Setup interface for creating a custom object. The breadcrumb trail at the top reads 'SETUP > OBJECT MANAGER'. The main heading is 'Policy'. On the left, a sidebar menu lists various configuration options: Details (selected), Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Search Layouts. The main content area is titled 'Policy' and contains a 'Custom Object Definition Edit' form. The form has three tabs: 'Custom Object Information' (active), 'Fields', and 'Relationships'. The 'Custom Object Information' tab contains the following fields and options: 'Label' (Policy), 'Plural Label' (Policies), 'Object Name' (Policy), and 'Description' (empty text area). There are also checkboxes for 'Starts with vowel sound' and 'Allow Search', and a 'Context-Sensitive Help Setting' section with two radio buttons: 'Open the standard Salesforce.com Help & Training window' (selected) and 'Open a window using a Visualforce page'.

Setup | Home | Object Manager ▾

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is always called "Name" when referenced via the API.

Record Name Example: Account Name

Data Type Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Display Format Example: A-{0000} [What is This?](#)

Optional Features

☒ Allow Reports

☒ Allow Activities

☒ Track Field History

☐ Allow in Chatter Groups

☐ Enable Licensing [i](#)

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

☒ Allow Sharing

☒ Allow Bulk API Access

☒ Allow Streaming API Access

Deployment Status [What is this?](#)

☐ In Development

☒ Deployed

Search Status

When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)

☒ Allow Search

Save Save & New Cancel

2. Configure Record Types on the Policy Object:

- Go to setup, Click on Object Manager.
- Search for Policy objects and click on Policy__c.
- In the left side panel, click on the record types.
- Click on New.
- Fill out the following details:
 - Select the existing record type as Master.
 - Give record type label as "Auto"
 - Record Type Name will be auto populated.
 - Give a meaningful description.
 - Check the active checkbox.
 - Check the Make Available checkbox for all the profiles.
 - Click on Next.
- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Edit Record Type

Auto

Enter a new name for the selected record type and click Save.

Record Type

Record Type Label: Auto

Record Type Name: Auto

Namespace Prefix:

Description:

Active: ☒

Save Cancel

Repeat the same steps for creating the property and life record types as well.

3. Configure Field Sets for Policy-Specific Data:

- Go to setup, Click on Object Manager.
- Search for Policy object and click on Policy__c.
- In the left side panel, click on the Field Sets.
- Click on New.
- Fill out the following details:
- Field Set Label as Vehicle
- Field Set Name will be Auto populated.
- Where is this used: In the policy object.
- Click on save.
- Click on the field set you have just created and it will redirect to the page layout setup.
- Drag VIN, and model year fields to the field set and click on save.

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Vehicle Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name *

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

VIN

Model Year

Repeat the same steps for creating the property and life Field sets as well.

3.1 Property FieldSet Requires:

- Square footage
- Year Built

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Property Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Square Footage

Year Built

3.2 Life FieldSet Requires:

- Beneficiary Name
- Policy Term Months

Setup Home Object Manager

SETUP > OBJECT MANAGER

Policy

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

Life Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Beneficiary Name

Policy Term Months

4. Add the Required Fields to the Policy Object:

S.No	Name of the Field	Data Type	Required
------	-------------------	-----------	----------

1	Beneficiary Name	Text(50)	
2	Customer	Lookup(Contact)	
3	Model Year	Text(5)	
4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium	Currency(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

5. Configure the Claim Custom Object:

- Go to setup, Click on object Manager.
- Click on create and select custom object.
- Fill the following details:
- Label: Claim
- Plural Label: Claim
- Object Name: Auto Populated
- Record Name: Claim Number
- Data type: Auto Number
- Format: C - {0000}
- Starting Number: 1
- Under optional features:
- Allow Reports, Activities and Track field History
- Under Deployment Status:
- Select the deployed radio button.
- Under Search Status:
- Check the Allow Search checkbox.
- Click on save.

Setup Home Object Manager

SETUP > OBJECT MANAGER

Claim

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Custom Object Information

The singular and plural labels are used in tabs, page layouts, and reports. Be careful when changing the name or label as it may affect existing integrations and merge templates.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Description

Context-Sensitive Help Setting ☒ Open the standard Salesforce.com Help & Training window ☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the

Setup Home Object Manager

SETUP > OBJECT MANAGER

Claim

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Record Name Example: Account Name

Data Type Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Display Format Example: A - {0000} What is This?

Optional Features

☒ Allow Reports

☒ Allow Activities

☒ Track Field History

☐ Allow in Chatter Groups

☐ Enable Licensing [i](#)

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

☒ Allow Sharing

☒ Allow Bulk API Access

☒ Allow Streaming API Access

Deployment Status

☐ In Development [What is this?](#)

☒ Deployed

6. Configure Record Types on the Claim Object:

- Go to setup, Click on Object Manager.
- Search for Claim object and click on Claim__c.
- In the left side panel, click on the record types.
- Click on New.
- Fill out the following details:
- Select the existing record type as Master.
- Give record type label as "Accident"
- Record Type Name will be auto populated.
- Give a meaningful description.
- Check the active checkbox.
- Check the Make Available checkbox for all the profiles.
- Click on Next.

- It will redirect to the Assign page layouts page, Leave everything as is, and click on save.

The screenshot shows the Salesforce Object Manager interface for the 'Claim' object. The left sidebar lists various configuration options, with 'Record Types' highlighted. The main content area is titled 'Edit Record Type: Accident'. Below the title, there is a prompt to 'Enter a new name for the selected record type and click Save.' The form contains the following fields:

- Record Type Label:** A text field containing 'Accident'.
- Record Type Name:** A text field containing 'Accident'.
- Namespace Prefix:** A text field that is currently empty.
- Description:** A text area that is currently empty.
- Active:** A checkbox that is checked.

At the bottom right of the form, there are 'Save' and 'Cancel' buttons. A 'Help for this Page' link is also visible in the top right corner.

//Repeat the same steps for creating the property and life record types as well.

7. Add the Required Fields to the Claim Object:

S.No	Field Name	Data Type	Required
1	Adjuster	Lookup(User)	
2	Approval Status	Picklist(New, Submitted for Approval, Approved, Rejected)	
3	Claim Amount	Currency(16, 2)	
4	Date of Loss	Date/Time	
5	Description	Long Text Area	
6	Policy	Lookup(Policy)	

8. Build the AutoQuoting Screen Flow

- Go to setup, Search for flows in the quick find box.
- Click on flows and click on new flow.
- Select Screen Flow and Click on Create.
- Add Get Records Element immediately after the start element.

- Give Label as **Get Vehicle RT ID**
- Api Name will be auto populated
- Select Record Type for Object
- For Conditional Requirements, Select All Conditions are met
- The Conditions include:
- sObject Type Name Equals Policy__c
- And
- Record Type Name Equals Auto

The screenshot shows the Salesforce Flow Builder interface for a flow named 'AutoQuotingFlow - V2'. The flow canvas displays a sequence of steps: 'Get Vehicle RT ID' (Get Records), 'New Auto Policy Quote - Basic Information' (Screen), 'Vehicle-Specific Details' (Screen), 'Create Draft Policy' (Create Records), and 'Calculate Premium Action' (Apex Action). The 'Get Vehicle RT ID' step is selected, and its configuration panel is open on the right. The configuration includes:

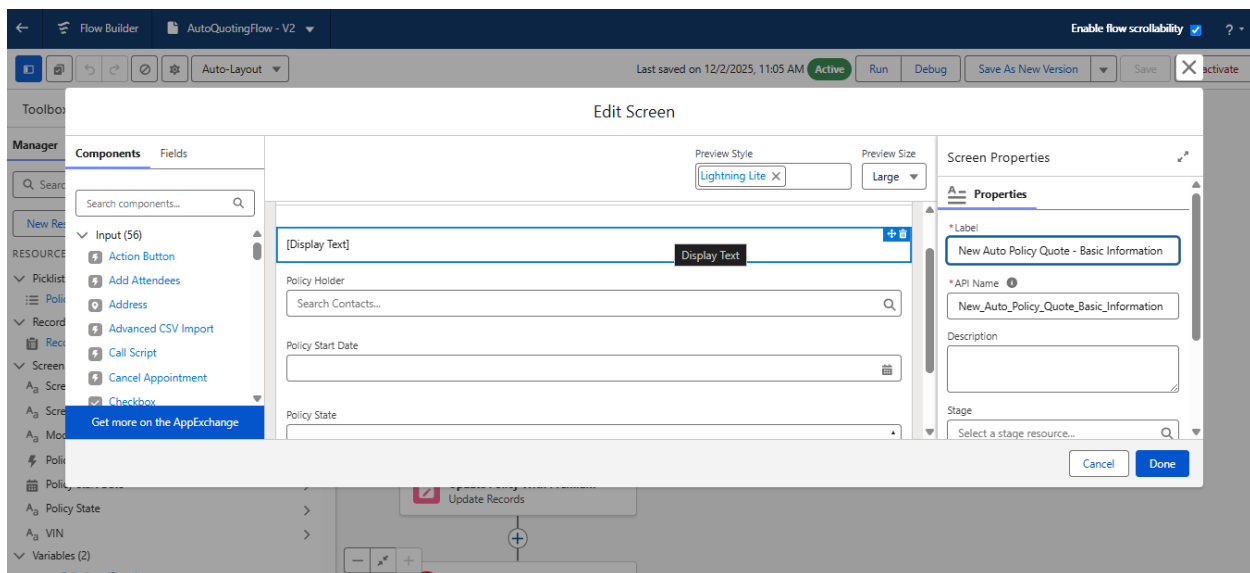
- *Label:** Get Vehicle RT ID
- *API Name:** Get_Vehicle_RT_ID
- Description:** (Empty text area)
- Get Records of This Object:** Record Type
- Filter Record Type Records:**
 - Condition Requirements:** All Conditions Are Met (AND)
 - Field 1:** sObject Type Name, **Operator:** Equals, **Value:** Policy__c
 - Field 2:** Record Type Name, **Operator:** Equals, **Value:** Auto

This screenshot shows the same 'Get Vehicle RT ID' step configuration panel, but with additional options visible at the bottom:

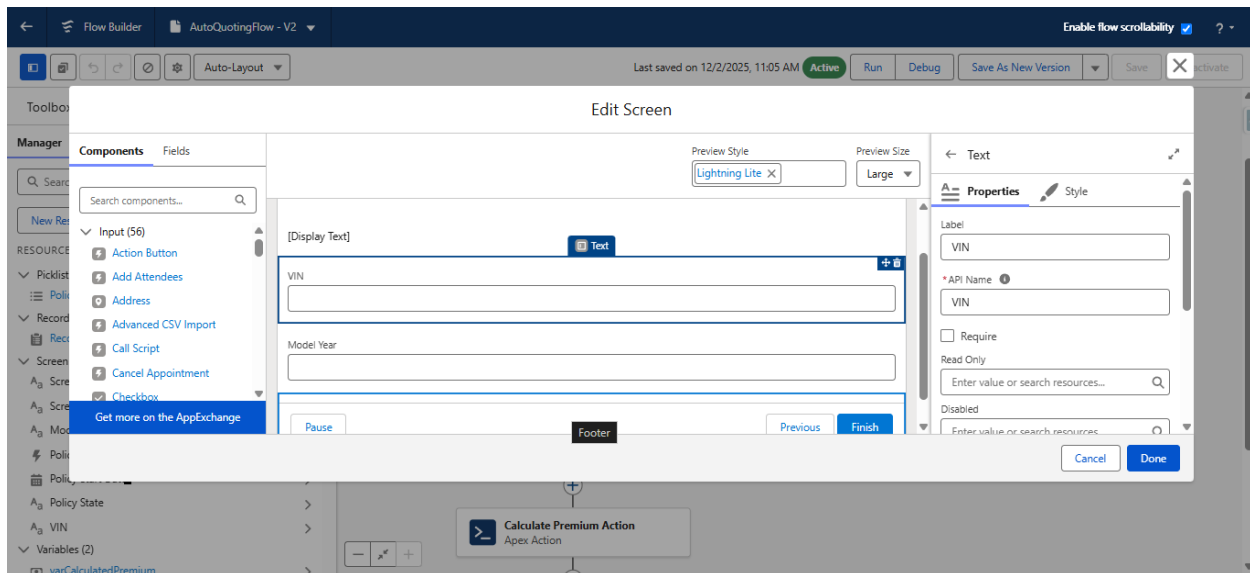
- Sort Record Type Records:**
 - Sort Order:** Not Sorted
 - How Many Records to Store:** Only the first record (selected)
 - How to Store Record Data:** Automatically store all fields (selected)

- Add Screen Element immediately after the Get Records Element
- Give the label as New Auto Policy Quote - Basic Information
- API Name will be auto populated.
- Drag the display text component into the flow canvas

- Give the API Name as Screen_1_Policy_Basics
- Drag the Lookup Component into the Canvas, Give API Name as Customer Lookup, Field API Name as Customer__c, Label as Policy Holder, ObjectApiName as Policy__c
- Drag Date component into the canvas, Give Label as Policy Start Date, API Name will be auto populated.
- Drag the picklist component into the canvas, Give the label as Policy State, API Name will be auto populated, For Choice, Create a new Resource, Resource Type should be Picklist Choice Set, Give the API Name, Select the Policy for Object, Select Picklist as Datatype, Select Policy State as Field and Click on save
- Click on Done



- Add another screen element after the Auto Policy Quote - Basic Information screen
- Give the label as Vehicle-Specific Details
- API Name will be auto populated.
- Drag the display text component into the flow canvas
- Give the API Name as Screen_2_Vehicle_Details
- Drag the text component into the canvas, Give the label as VIN, API name will be auto populated
- Again, Drag the text component into the canvas, Give the label as Model year, API name will be auto populated.
- Click on done.



- Add Create Records Element after the screen element.
- Give the label as **Create Draft Policy**, API name will be auto populated
- Store field values manually
- Select the policy object
- Set fields for the policy object as shown below:
- Customer —> From Basic information screen-> Policy Holder -> Record ID
- Model Year——> Vehicle information screen -> Model Year
- Policy Start Date —> From Basic information screen→Policy Start Date
- Policy State —> From Basic information screen —> Policy State
- RecordType ID —> From Get Vehicle RT Id —> RecordType ID
- VIN —> From Vehicle Information Screen —> VIN
- Check manually assign variables checkbox
- In the place of variable, create a new resource, Whose resource type is variable, API Name would be varPolicyId, Datatype would be Text, Click on Done.

The top screenshot shows the Salesforce Flow Builder interface for the 'AutoQuotingFlow - V2'. The flow is active and saved on 12/2/2025 at 11:05 AM. The flow consists of several steps: 'New Auto Policy Quote - Basic Information' (Screen), 'Vehicle-Specific Details' (Screen), 'Create Draft Policy' (Create Records), 'Calculate Premium Action' (Apex Action), and 'Update Policy With Premium' (Update Records). The 'Create Draft Policy' action is selected, and its configuration panel is visible on the right. The panel shows the label 'Create Draft Policy', the API Name 'Create_Draft_Policy', and the object 'Policy'. The 'How to set record field values' is set to 'Manually'. The 'Set Field Values for the Policy' section shows two fields: 'Customer' and 'Model Year', both mapped to values from the 'Basic Information' screen.

The bottom screenshot shows the same flow, but the 'Create Records' action is now configured to create a record of the 'Policy' object. The 'Set Field Values for the Policy' section shows five fields: 'Model Year', 'Policy Start Date', 'Policy State', 'Record Type ID', and 'VIN'. Each field is mapped to a value from a screen or variable in the flow. The 'Manually assign variables (advanced)' checkbox is checked, and the 'Store Policy ID in Variable' section shows the variable 'varPolicyId' being used to store the Policy ID. The 'Check for Matching Records' checkbox is disabled.

- Save the flow with AutoQuotingFlow
- Activate the flow

9. Add a Validation Rule to the Policy Object:

- Go to object manager, Search for the policy object and click on policy object
- In the left side panel, scroll down and find validation rules and click on it
- Click on New

SETUP > OBJECT MANAGER

Policy

Record Types
Related Lookup Filters
Search Layouts
List View Button Layout
Restriction Rules
Scoping Rules
Object Access
Triggers
Flow Triggers
Validation Rules
Conditional Field Formatting

Validation Rules
1 Items, Sorted by Rule Name

[New](#)

RULE NAME	ERROR LOCATION	ERROR MESSAGE	ACTIVE	MODIFIED BY
VIN_Must_Be_17_Characters	VIN	The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.	✓	Insurance Portal, 12/1/2025, 8:50 PM

<https://orgfarm-2d73826e88-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000003OBi9/ValidationRules/03dgK000000YHFGAW/view>

- Give the rule name as VIN_Must_Be_17_Characters
- Check the active checkbox
- Write the following formula in the formula editor
 - `AND( RecordType.DeveloperName = "Auto", LEN( VIN__c )<> 17)`
- For error message, The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Select error location as field and select VIN field
- Click on Save

SETUP > OBJECT MANAGER

Policy

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts

Policy Validation Rule
[Back to Policy](#)

Validation Rule Detail [Edit](#) [Clone](#)

Rule Name: VIN_Must_Be_17_Characters Active: ✓

Error Condition Formula: `AND( RecordType.DeveloperName = "Auto", LEN( VIN__c )<> 17)`

Error Message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies. Error Location: VIN

Description: [Insurance Portal](#), 12/1/2025, 8:50 PM Modified By: [Insurance Portal](#), 12/1/2025, 8:50 PM

[Edit](#) [Clone](#)

Milestone 2: Policy Issuance and Automated Claim Routing

1. Create the PremiumCalculator Apex class to calculate premium values from policy data

- Click on setup icon, Right click on developer console, Open the link in new tab
- At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as PremiumCalculator and click on create.
- Paste the following code in the apex class:

```
public class PremiumCalculator {
```

```
    public class PremiumInputWrapper {
```

```
        @InvocableVariable(required=true)
```

```
        public Id policyId;
```

```
    }
```

```
    @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on policy fields (VIN, Model Year, State).')
```

```
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
```

```
        List<Decimal> premiumResults = new List<Decimal>();
```

```
        Set<Id> policyIds = new Set<Id>();
```

```
        for (PremiumInputWrapper input : inputs) {
```

```
            if (input.policyId != null) {
```

```
policyIds.add(input.policyId);
```

```
}
```

```
}
```

```
List<Policy__c> policiesToRate = [
```

```
    SELECT Id, VIN__c, Model_Year__c, Policy_State__c
```

```
    FROM Policy__c
```

```
    WHERE Id IN :policyIds
```

```
];
```

```
for (Policy__c policy : policiesToRate) {
```

```
    Decimal basePremium = 1000.00;
```

```
    if (policy.Policy_State__c == 'CA') {
```

```
        basePremium *= 1.15;
```

```
    } else if (policy.Policy_State__c == 'TX') {
```

```
        basePremium *= 1.05;
```

```

    }

    if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) <
2018) {

        basePremium += 100;

    }

    premiumResults.add(basePremium);

}

return premiumResults;

}

}

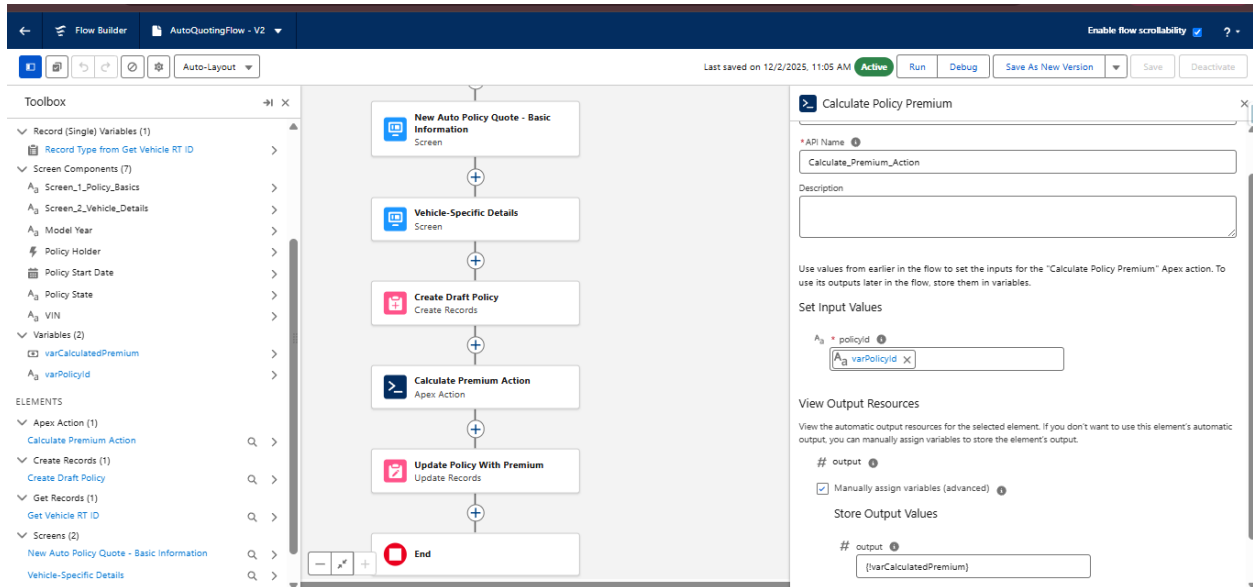
```

- Click on CTRL + S to save the apex class
- And this apex class need to be invoked in the AutoQuotingFlow

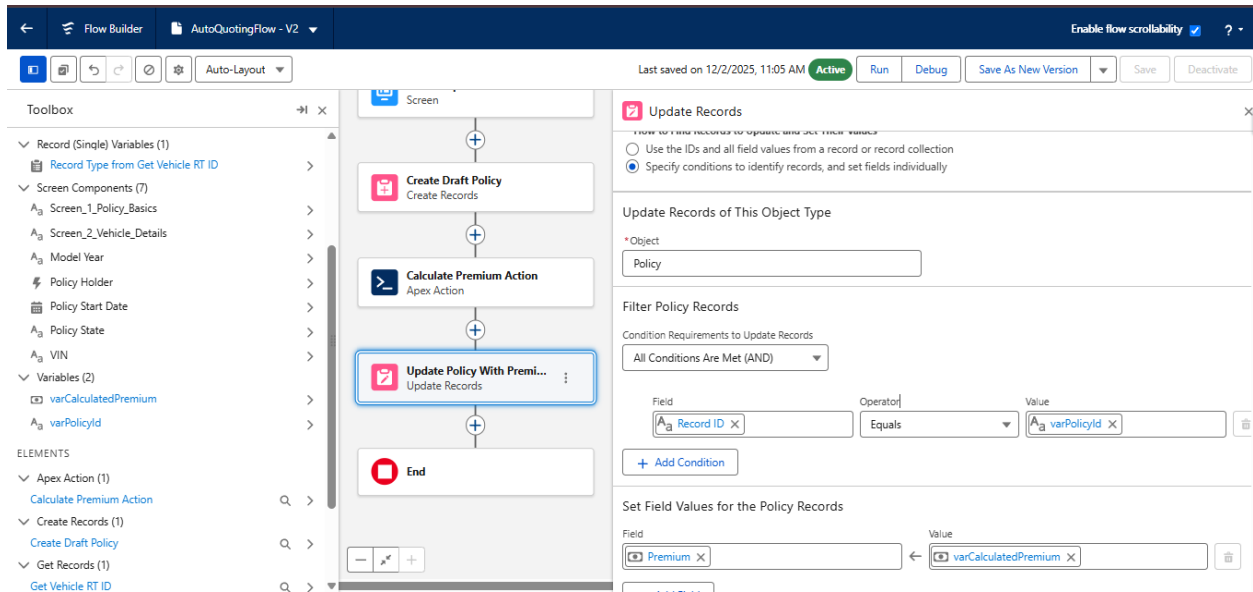
2. Integrate the Apex action with the final stage of AutoQuotingFlow so the draft Policy record is updated with the calculated premium

- Go to setup, in the quick find box search for flows and click on flows
- Find for AutoQuotingFlow and open in the flow builder
- Under the Create Records element,
- Add an action element after the create records element.

- Give the label as Calculate Premium Action
- API Name will be auto populated
- Under set Input values, For policy Id, Set the varPolicyId variable created in the previous step
- Under view output resources, check the manually assign variables checkbox and for the output placeholder, create a resource
- Resource type would be variable, API Name would be varCalculatedPremium, Data type would be currency, Decimal places are '2' and click Done.



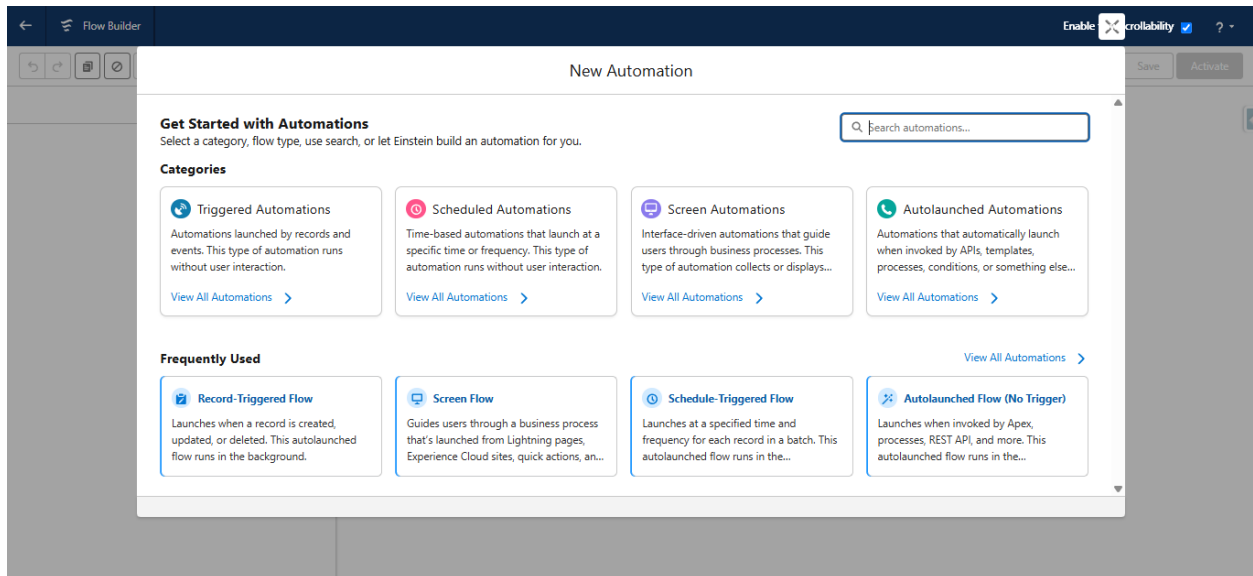
- After the action element, add the update records element, Give the label as Update Policy With Premium
- For how to find records to update and set their values, Select Specify conditions to identify records, and set fields individually
- Select object as policy
- Select all conditions are met for filter criteria
- The conditions include:
 - Record Id Equals varPolicyId variable
- Under set field values for policy record
- Assign varCalculatedPremium variable to the Premium field



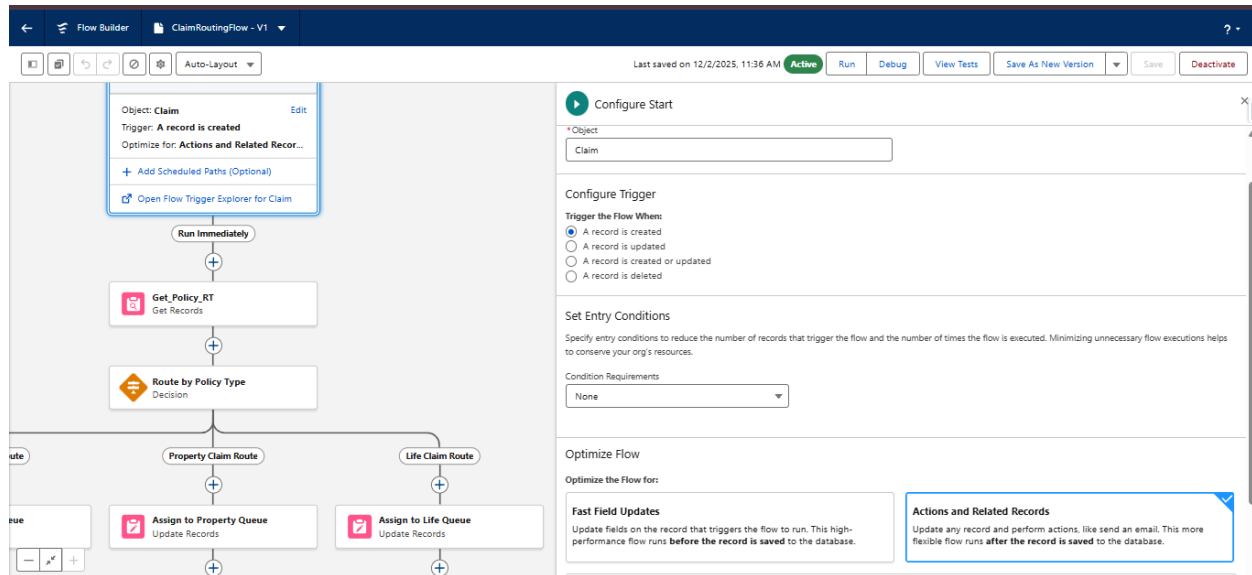
- Save the flow as a new version
- Activate the flow.

3. Create an After-Save Record-Triggered Flow on the Claim object for automated routing.

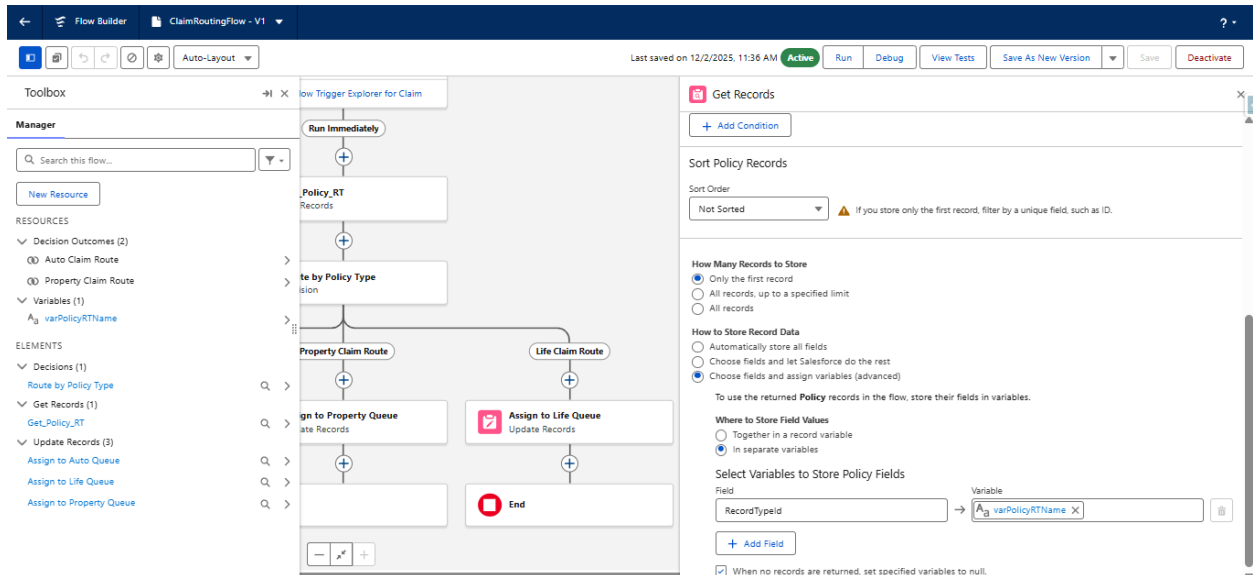
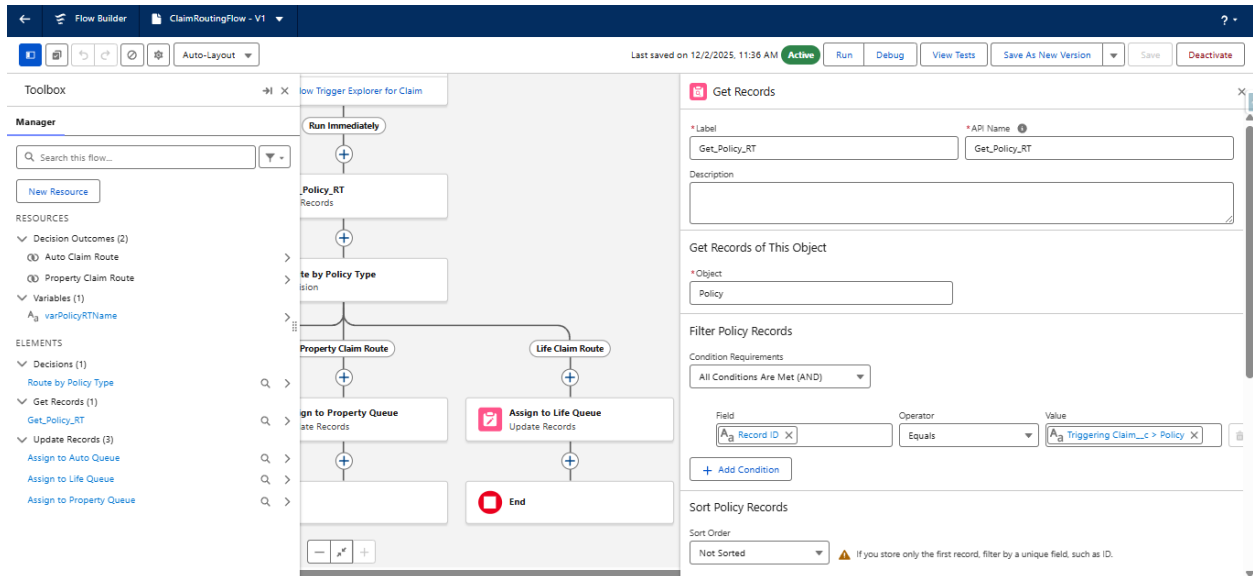
- Go to setup, In the quick find box, Search for flows and click on flows
- Click on New Flow, select Record Triggered Flow and Click on Create



- Select Claim for an object
- Configure trigger when the record is created
- Optimize flow for action and Related Records



- Add Get Records element after the start and give the label as Get_Policy_RT
- Select policy for object
- Specify all conditions are met for conditional requirements
- The conditions include:
 - Record Id Equals Triggering Claim → Policy
 - Store only the first record
 - Choose fields and assign variables for storing record data
 - Select in Separate variables for where to store field values section
 - Under select variables to store policy fields
- For recordTypeID field Create a resource, Resource type would be variable, API Name would be varPolicyRTName, Data type would be Text and click on Done



- Under Get Records Element, Add decision element
- Give the label as Route by Policy Type
- API name will be auto populated
- Select Decision Logic as Manually set conditions
- Specify the Outcomes:
- Outcome1:
 - Label as Auto Claim Route
 - API name will be auto populated
 - Select All conditions for the conditional Requirements
 - The condition includes:
 - varPolicyRTName variable equals Auto
- Outcome2:

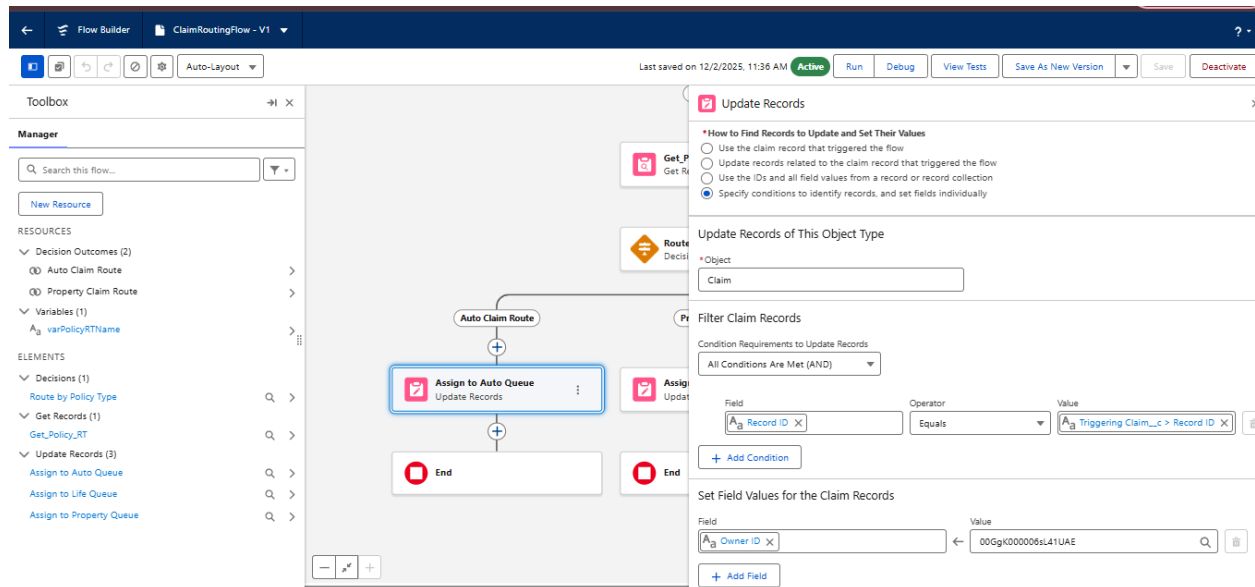
- Label as Property Claim Route
 - API name will be auto populated
 - Select All conditions for the conditional Requirements
 - The condition includes:
 - varPolicyRTName variable equals Property
- Default Outcome:
 - Label as Life Claim Route

The screenshot shows the Salesforce Flow Builder interface for a flow named 'ClaimRoutingFlow - V1'. The flow diagram on the left shows the sequence: Run Immediately → Get_Policy_RT (Get Records) → Route by Policy Type (Decision) → Property Claim Route → Assign to Property Queue (Update Records) → End. The right-hand pane is focused on the 'Route by Policy Type' decision element. It shows two outcomes: 'Auto Claim Route' and 'Property Claim Route'. The 'Auto Claim Route' outcome is selected, and its configuration is displayed: Outcome Label is 'Auto Claim Route', Outcome API Name is 'Auto_Claim_Route', and the condition is 'varPolicyRTName Equals Auto'. The 'Property Claim Route' outcome is also visible but not selected.

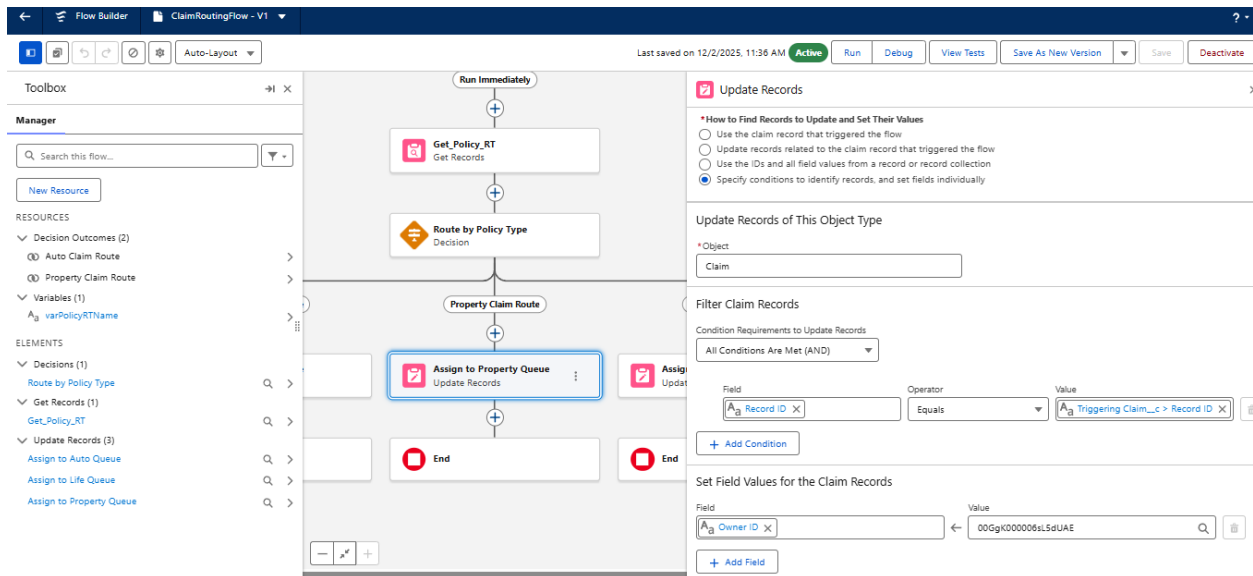
This screenshot shows the same Salesforce Flow Builder interface, but now the 'Property Claim Route' outcome is selected in the right-hand pane. The configuration for this outcome is shown: Outcome Label is 'Property Claim Route', Outcome API Name is 'Property_Claim_Route', and the condition is 'varPolicyRTName Equals Property'. The 'Auto Claim Route' outcome is still visible but not selected.

- Under Auto Claim Route Decision path, add the update records element
- Label it as Assign to Auto Queue
- API name will be auto populated

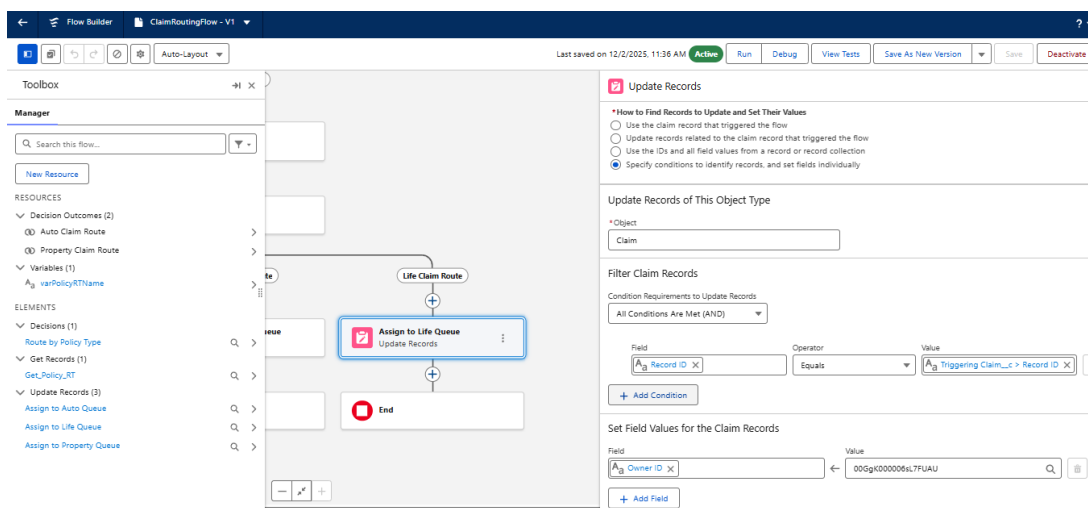
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded auto queue 18 digit Id for Owner ID field



- Under Property Claim Route Decision path, add the update records element
- Label it as Assign to Property Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Property queue 18 digit Id for Owner ID field



- Under Life Claim Route Decision path, add the update records element
- Label it as Assign to Life Queue
- API name will be auto populated
- Under How to find records to update and set their fields, select specify conditions to identify records
- Select Claim as an object
- Select All conditions are true for Conditional requirements
- The conditions include:
- Record Id equals Triggering Claim → Record ID
- Under Set fields for the Claim Records
- Set the hardcoded Life queue 18 digit Id for Owner ID field



Milestone 3: Claims Adjuster Dashboard using LWC

1. **Create the ClaimsAdjusterController Apex class. Use SOQL to retrieve claims owned by the logged-in user together with related Policy and Customer information, and return the required claim metrics through a custom wrapper class.**

- Click on setup icon, Right click on developer console, Open the link in new tab
- At the top left corner, click on the file, then click on new, and click on Apex class, Give the name as ClaimsAdjusterController and click on create.
- Paste the following code in the apex class:

```
public class ClaimsAdjusterController {
```

```
    public class ClaimWrapper {
```

```
        @AuraEnabled public Id claimId;
```

```
        @AuraEnabled public String claimNumber;
```

```
        @AuraEnabled public Decimal claimAmount;
```

```
        @AuraEnabled public String status;
```

```
        @AuraEnabled public String policyType;
```

```
        @AuraEnabled public String policyHolderName;
```

```
        @AuraEnabled public Integer daysOpen;
```

```
    public ClaimWrapper(Claim__ c c, Policy__ c p, Contact customerRec) {
```

```
        this.claimId = c.Id;
```

```
this.claimNumber = c.Name;
```

```
this.claimAmount = c.Claim_Amount__c;
```

```
this.status = c.Approval_Status__c;
```

```
Policy__c policyRecord = (Policy__c)p;
```

```
if (policyRecord != null && policyRecord.RecordType != null) {
```

```
    this.policyType = policyRecord.RecordType.Name;
```

```
    } else {
```

```
        this.policyType = 'Unknown';
```

```
    }
```

```
this.policyHolderName = (customerRec != null) ? customerRec.Name : "";
```

```
this.daysOpen = (c.Date_of_Loss__c != null)
```

```
    ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
```

```
    : 0;
```

```
}
```

```
}
```

```
@AuraEnabled(cacheable=true)
```

```
public static List<ClaimWrapper> getAssignedClaims() {
```

```
|
```

```
List<ClaimWrapper> results = new List<ClaimWrapper>();
```

```
|
```

```
List<Claim__c> claims = [
```

```
SELECT
```

```
Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
```

```
Policy__r.Name,
```

```
Policy__r.RecordType.DeveloperName,
```

```
Policy__r.Customer__r.FirstName,
```

```
Policy__r.Customer__r.LastName
```

```
FROM Claim__c
```

```
WHERE OwnerId = :UserInfo.getUserId()
```

```
ORDER BY Date_of_Loss__c DESC
```

```
];
```

```
for (Claim__c claim : claims) {
```

```
    Policy__c policy = claim.Policy__r;
```

```
    Contact customer = (policy != null && policy.Customer__r != null)
```

```
        ? (Contact)policy.Customer__r
```

```
        : new Contact();
```

```
    results.add(new ClaimWrapper(claim, policy, customer));
```

```
}
```

```
return results;
```

```
}
```

```
}
```

- Save the file using CTRL + S
- This apex class will be handled in the LWC component

2. Create the claimsDashboardLwc component, connect it to the Apex method through @wire, and provide client-side filtering for policy type.

- Open the new tab in chrome and search for lightning studio extension
- Open the first link and click on add to chrome, and confirm by clicking add Extension button
- Now open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as claimsDashboardLwc
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```
<template>

    <lightning-card title="My Assigned Claims Dashboard" icon-
name="standard:claim">

        <div class="slds-m-around_medium">

            <lightning-combobox

                name="policyFilter"

                label="Filter by Policy Type"

                value="All"

                options={policyTypeOptions}
```

```

        onchange={handleFilterChange}

        class="slds-m-bottom_medium"

    ></lightning-combobox>

    <template if:true={visibleClaims}>

        <div class="slds-grid slds-wrap slds-grid_pull-padded">

            <template for:each={visibleClaims} for:item="claim">

                <div key={claim.claimId} class="slds-col slds-
size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3 slds-p-
horizontal_small">

                    <c-claim-tile-lwc claim-data={claim}></c-
claim-tile-lwc>

                </div>

            </template>

        </div>

        <p class="slds-m-top_medium slds-text-heading_small">Total
Claims Assigned: {visibleClaims.length}</p>

    </template>

    <template if:false={visibleClaims}>

```

```
        <p>No claims currently assigned to you.</p>

    </template>

</div>

</lightning-card>

</template>
```

- Open the JS file and paste the following code:

```
import { LightningElement, wire, track } from 'lwc';

// Import the Apex method

import getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {

    // List to hold all data retrieved from Apex

    @track allClaims = [];

    // List to display after filtering

    @track visibleClaims = [];
```

```
error;

// Wire service to call the Apex method

@wire(getAssignedClaims)

wiredClaims({ error, data }) {

    if (data) {

        this.allClaims = data;

        this.visibleClaims = data; // Initially display all claims

        this.error = undefined;

    } else if (error) {

        this.error = error;

        this.allClaims = undefined;

        this.visibleClaims = undefined;

        console.error('Error retrieving claims:',
JSON.stringify(error));

    }

}
```

```
// Client-side filtering logic (Required by M3)
```

```
handleFilterChange(event) {
```

```
    const selectedPolicyType = event.target.value;
```

```
    if (selectedPolicyType === 'All') {
```

```
        this.visibleClaims = this.allClaims;
```

```
    } else {
```

```
        // Filter claims where policyType matches the selected value
```

```
        this.visibleClaims = this.allClaims.filter(claim =>
```

```
            claim.policyType === selectedPolicyType
```

```
        );
```

```
    }
```

```
}
```

```
get policyTypeOptions() {
```

```

// Options for the filter dropdown

return [

    { label: 'All Policy Types', value: 'All' },

    { label: 'Auto', value: 'Auto' },

    { label: 'Property', value: 'Property' },

    { label: 'Life', value: 'Life' }

];

}

}

```

- Open the meta-xml file and paste the following code:

```

<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

    <targets>

```

```

<target>lightning__AppPage</target>

<target>lightning__HomePage</target>

</targets>

</LightningComponentBundle>

```

3. Create a reusable claimTileLwc component to present the key details of each claim in a compact format.

- Open the Lightning studio extension from your salesforce org
- On the Top left corner, you will find Create New
- Click on Create New and Click on Lightning Web Component
- Enter the name of the Component as **claimTileLwc**
- Check the isExposed checkbox
- Under target configurations, Select Lightning app page, Home page and Record page
- Click on Deploy
- Open the HTML file and paste the following code:

```

<template>

  <lightning-layout-item size="12" padding="around-small">

    <div class="tile slds-box slds-p-around_medium slds-theme_default">

      <header class="slds-media slds-media_center slds-m-bottom_small">

        <div class="slds-media__figure">

          <lightning-icon icon-name={policyIcon} alternative-text={claimData.policyType} size="small"></lightning-icon>

```

```
</div>

<div class="slds-media__body slds-m-left_small">

    <h2 class="slds-text-heading_medium slds-m-
bottom_none">

        Claim: {claimData.claimNumber}

    </h2>

    <p class="slds-text-body_small">Policy Type:
<strong>{claimData.policyType}</strong></p>

</div>

</header>

<div class="slds-border_top slds-p-top_small">

    <p>Policy Holder: <strong>{claimData.policyHolderName}
</strong></p>

    <lightning-formatted-number

        value={claimData.claimAmount}

        format-style="currency"
```

```

        currency-code="USD"

        label="Claim Amount">

    </lightning-formatted-number>

    <p>Status: {claimData.status}</p>

    <p class="slds-text-body_regular">

        Days Open: <strong class="slds-text-
color_error">{claimData.daysOpen}</strong>

    </p>

</div>

</div>

</lightning-layout-item>

</template>

```

- Open the JS File and paste the following code

```
import { LightningElement, api } from 'lwc';
```

```
export default class ClaimTileLwc extends LightningElement {  
  
    // @api decorator exposes this property to the parent component  
  
    @api claimData;  
  
    // Helper property to determine the correct icon based on Policy Type  
  
    get policyIcon() {  
  
        if (this.claimData && this.claimData.policyType) {  
  
            const type = this.claimData.policyType.toLowerCase();  
  
            if (type === 'auto') return 'utility:truck';  
  
            if (type === 'property') return 'utility:home';  
  
            if (type === 'life') return 'utility:connected_apps';  
  
        }  
  
        return 'utility:help';  
  
    }  
  
}
```

- Open the meta-xml file and paste the following code

```
<?xml version="1.0"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>63.0</apiVersion>

    <isExposed>true</isExposed>

</LightningComponentBundle>
```

Milestone 4: Advanced Claim Processing, Security, and Testing

1. Enhance the Claim object by adding an Approval Status field.

- Go to set up, Click on object manager, search for claim object and click on claim object.
- In the left side panel, click on fields and relationships.
- Click on new
- Select Datatype as picklist
- Label it as Approval Status
- Select enter values radio button
- Enter New, Submitted for Approval, Approved and Rejected line by line
- Field Name will be auto populated
- Click on Next, Next and Save.

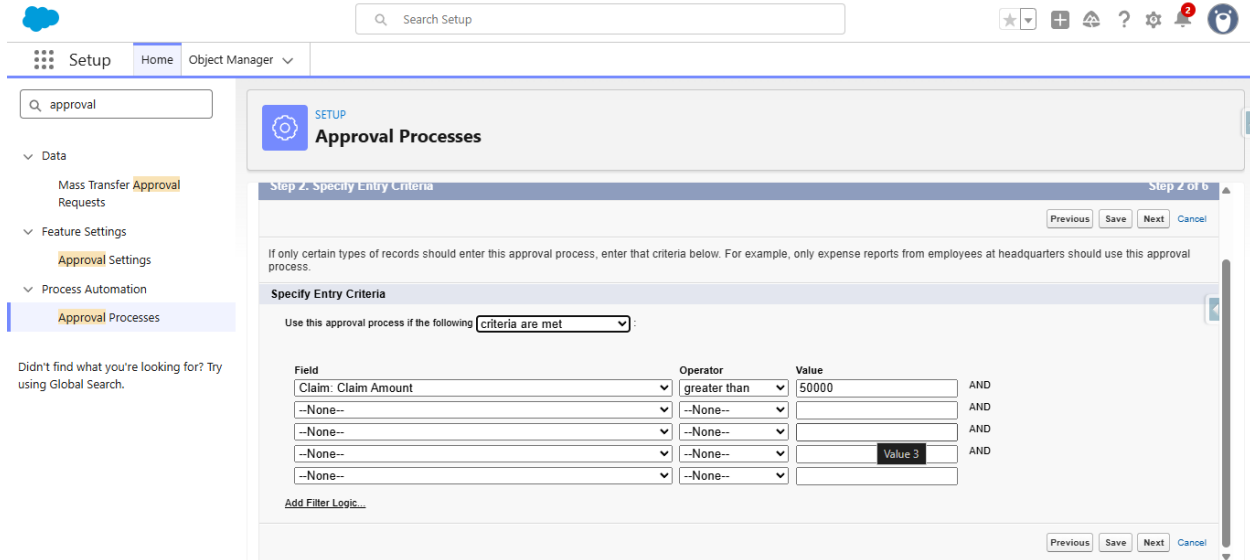
2. Configure a two-stage Approval Process that first routes the claim to the Senior Adjuster and then to the Department Manager.

- Go to setup, In the quick find box, Search for Approval process

- Select claim object for Manage approval process for
- Click on create new Approval Process and select use standard setup wizard
- Give the process name as High Value Claim Approval
- Unique Name will be auto populated
- Click on Next

The screenshot displays the Salesforce Setup interface for configuring an approval process. The main heading is 'Approval Processes'. Below it, the wizard title is 'High Value Claim Approval'. The current step is 'Step 1 of 6: Enter Name and Description'. The form contains three fields: 'Process Name' with the value 'High Value Claim Approval', 'Unique Name' with the value 'High_Value_Claim_Approval', and an empty 'Description' field. A red error message 'Required Information' is visible next to the 'Unique Name' field. The sidebar on the left shows the navigation menu with 'Approval Processes' selected. The top navigation bar includes 'Setup', 'Home', and 'Object Manager'.

- Under specify entry criteria, Select criteria are met for Use this approval process if
- Criteria includes:
- Claim amount greater than 50000
- Click on Next



Search Setup

Setup Home Object Manager

approval

- Data
 - Mass Transfer **Approval** Requests
- Feature Settings
 - Approval** Settings
- Process Automation
 - Approval** Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Step 2: Specify Entry Criteria

Previous Save Next Cancel

If only certain types of records should enter this approval process, enter that criteria below. For example, only expense reports from employees at headquarters should use this approval process.

Specify Entry Criteria

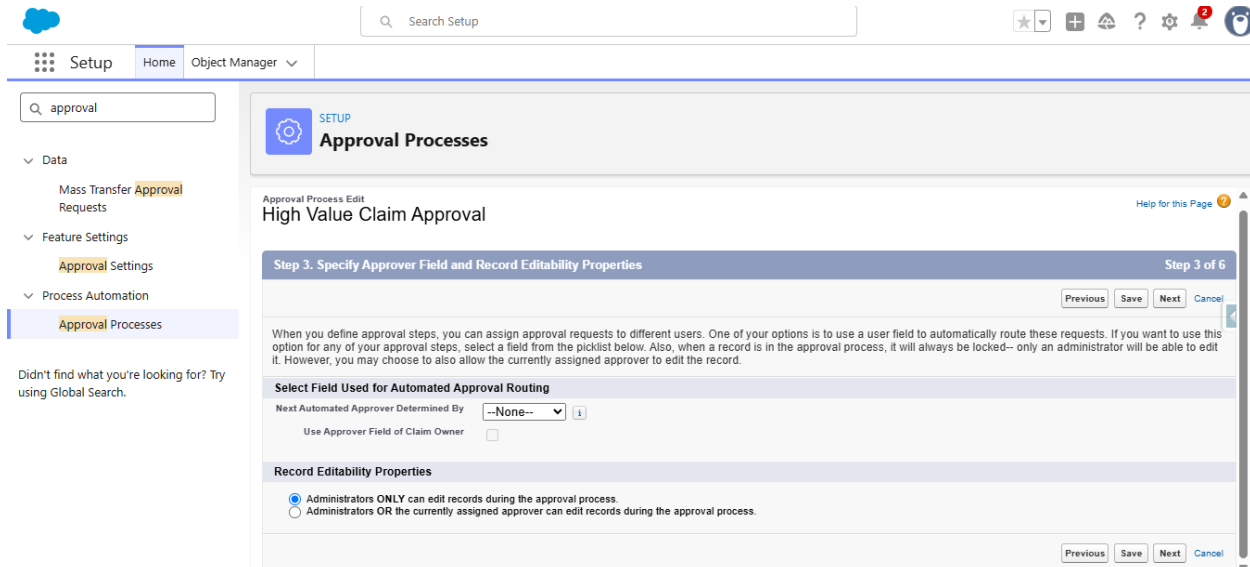
Use this approval process if the following criteria are met:

Field	Operator	Value	
Claim: Claim Amount	greater than	50000	AND
--None--	--None--		AND
--None--	--None--		AND
--None--	--None--	Value 3	AND
--None--	--None--		

Add Filter Logic...

Previous Save Next Cancel

- Under Record Editability properties, Select Administrators ONLY can edit the records during the approval process
- Click on Next



Search Setup

Setup Home Object Manager

approval

- Data
 - Mass Transfer **Approval** Requests
- Feature Settings
 - Approval** Settings
- Process Automation
 - Approval** Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Approval Process Edit

High Value Claim Approval

Step 3: Specify Approver Field and Record Editability Properties

Previous Save Next Cancel

When you define approval steps, you can assign approval requests to different users. One of your options is to use a user field to automatically route these requests. If you want to use this option for any of your approval steps, select a field from the picklist below. Also, when a record is in the approval process, it will always be locked-- only an administrator will be able to edit it. However, you may choose to also allow the currently assigned approver to edit the record.

Select Field Used for Automated Approval Routing

Next Automated Approver Determined By: --None-- ⓘ

Use Approver Field of Claim Owner: ☐

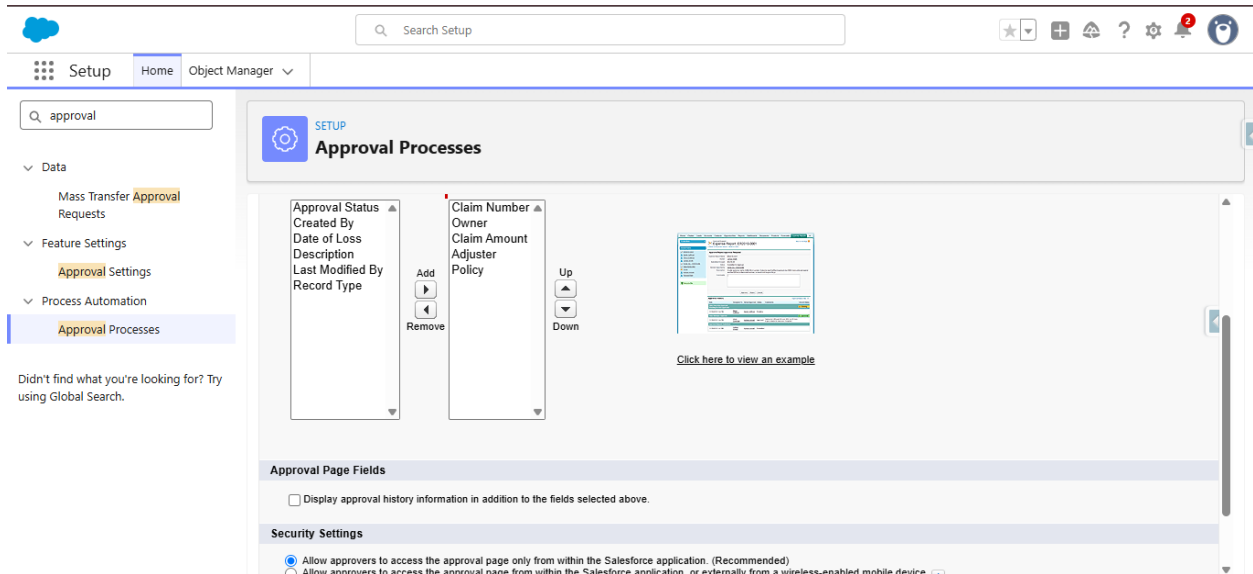
Record Editability Properties

☒ Administrators ONLY can edit records during the approval process.

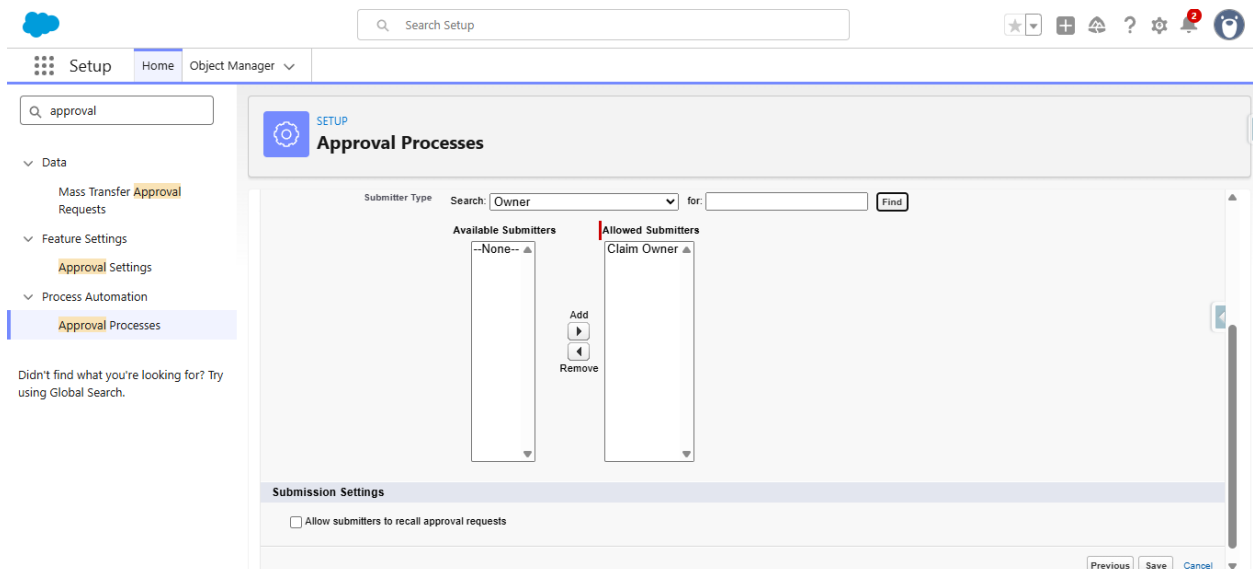
☐ Administrators OR the currently assigned approver can edit records during the approval process.

Previous Save Next Cancel

- Click on next again
- Under Approval Page Layout, Select Claim Number, Owner, Claim Amount, Adjuster, Policy fields to the selected fields from available fields
- Under security settings, Select Allow Approvers to access the page only from within the salesforce application.
- Click on Next



- Under Initial Submitters, Search for owner under submitter type, and select the claim owner as allowed submitters from available submitters
- Click on Save



2.1 Create Approval Steps:

Step1:

- Under initial submission actions, You can find Approval Steps
- Under approval steps, Click on New

- Give the Name as Step1 Senior Adjuster
- Unique name will be auto populated
- Click on Next

The screenshot shows the Salesforce Setup interface. The left sidebar contains a search bar with 'approval' and a navigation menu with categories: Data, Feature Settings, and Process Automation. Under Process Automation, 'Approval Processes' is selected. The main content area is titled 'Approval Processes' and 'Step1 Senior Adjuster'. It shows 'Step 1 of 3' with the heading 'Step 1. Enter Name and Description'. Below this, there is a text input field for 'Enter a name, description, and step number for your new approval step.' and a section titled 'Enter Name and Description'. This section includes fields for 'Approval Process Name' (High Value Claim Approval), 'Name' (Step1 Senior Adjuster), 'Unique Name' (Step1_Senior_Adjuster), and 'Description'. A red error message indicates 'Required Information' is missing. Navigation buttons 'Save', 'Next', and 'Cancel' are at the bottom right.

- Under specify step criteria, Select all records should enter the step
- Click on Next

The screenshot shows the Salesforce Setup interface for 'Step 2 of 3' titled 'Specify Step Criteria'. The left sidebar is the same as the previous screenshot. The main content area shows 'Step 2. Specify Step Criteria' with a heading 'Specify whether a record must meet certain criteria before entering this approval step. If these criteria are not met, the approval process can skip to the next step, if one exists. [Learn more](#)'. Below this is a section titled 'Specify Step Criteria' with two radio button options: 'All records should enter this step.' (selected) and 'Enter this step if the following criteria are met'. The 'Enter this step if the following' option has dropdown menus for 'criteria are met' and 'reject record'. Navigation buttons 'Previous', 'Save', 'Next', and 'Cancel' are at the bottom right.

- Under select Approver, Select Automatically assign to approver
- Under that select the Senior Adjuster User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Click on Save

Setup Home Object Manager

Search Setup

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Step 3. Select Assigned Approver

Step 3 of 3

Previous Save Cancel

Specify the user who should approve records that enter this step. Optionally, choose whether the approver's delegate is also allowed to approve these requests.

Select Approver

☐ Let the submitter choose the approver manually.
☐ Automatically assign to queue.
☒ Automatically assign to approver(s).

User: Senior Adjuster
 Add Row Remove Row

When multiple approvers are selected:
☒ Approve or reject based on the FIRST response.
☐ Require UNANIMOUS approval from all selected approvers.

☐ The approver's delegate may also approve this request.

Previous Save Cancel

- Click on New again to create one more step
- Give the name as Step2 Manager Review
- Unique name will be auto populated
- Click on Next

Setup Home Object Manager

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Save Next Cancel

Enter a name, description, and step number for your new approval step.

Previous Approval Step Information	
Step Number:	1
Name:	Step1 Senior Adjuster
Criteria:	
Assign To:	User : Senior Adjuster

Enter Name and Description

Approval Process Name: High Value Claim Approval

Name: Step2 Manager Review

Unique Name: Step2_Manager_Review

Description:

Save Next Cancel

- Under specify step criteria, Select all records should enter the step
- Click on Next

Setup Home Object Manager

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Step 2. Specify Step Criteria

Step 2 of 3

Previous Save Next Cancel

Specify whether a record must meet certain criteria before entering this approval step. If these criteria are not met, the approval process can skip to the next step, if one exists. [Learn more](#)

Previous Approval Step Information	
Step Number:	1
Name:	Step 1 Senior Adjuster
Criteria:	
Assign To:	User : Senior Adjuster

Specify Step Criteria

☒ All records should enter this step.

☐ Enter this step if the following criteria are met

Previous Save Next Cancel

- Under select Approver, Select Automatically assign to approver
- Under that select the Department Manager User
- Under when multiple approvers are selected, select Approve or Reject based on the FIRST Response
- Under Reject behaviour, Select Perform all rejection actions for this step AND all final rejection actions
- Click on Save

Setup Home Object Manager

approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Select Approver

☐ Let the submitter choose the approver manually.

☐ Automatically assign to queue.

☒ Automatically assign to approver(s).

User	Claims Manager
Add Row Remove Row	

☒ When multiple approvers are selected: Approve or reject based on the FIRST response.

☐ Require UNANIMOUS approval from all selected approvers.

☐ The approver's delegate may also approve this request.

Reject Behavior

What should happen if the approver rejects this request?

☒ Perform all rejection actions for this step AND all final rejection actions. (Final Rejection)

☐ Perform ONLY the rejection actions for this step and send the approval request back to the most recent approver. (Go Back 1 Step)

Previous Save Cancel

- Under Initial Submission Actions, Click on Add New and select Field Update
- Give the name as Update Status Submitted
- Unique name will be auto populated
- Select object as Claim

- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select submitted for Approval
- Click on Save

The screenshot shows the Salesforce Setup interface for configuring a Field Update. The left sidebar contains a search bar and navigation links for Data, Feature Settings, and Process Automation. The main content area is titled 'Field Updates' and is divided into two sections: 'Identification' and 'Specify New Field Value'.

Identification Section:

- Name:** Update Status Submitted
- Unique Name:** Update_Status_Submitted
- Description:** (Empty text box)
- Object:** Claim
- Field to Update:** Claim: Approval Status
- Field Data Type:** Picklist
- Re-evaluate Workflow Rules after Field Change:** (Unchecked checkbox)

Specify New Field Value Section:

- Picklist Options:**
 - ☐ The value above the current one
 - ☐ The value below the current one
 - ☒ A specific value: Submitted for Approval

At the bottom of the page, there are three buttons: 'Save', 'Save & New', and 'Cancel'.

- Under Final Approval Actions, Click on Add New and select Field Update
- Give the name as Update Status Approved
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Approved
- Click on Save

The screenshot shows the Salesforce Setup interface with the 'Field Updates' configuration page. The left sidebar contains a search bar with 'approval' and a navigation menu with categories: Data (Mass Transfer Approval Requests), Feature Settings (Approval Settings), and Process Automation (Approval Processes). The main content area is titled 'Field Updates' and includes a 'Required Information' icon. The 'Identification' section contains the following fields: Name (Update Status Approved), Unique Name (Update_Status_Approved), Description (empty), Object (Claim), Field to Update (Approval Status), Field Data Type (Picklist), and a checkbox for 'Re-evaluate Workflow Rules after Field Change' (unchecked). The 'Specify New Field Value' section shows 'Picklist Options' with three radio buttons: 'The value above the current one' (unchecked), 'The value below the current one' (unchecked), and 'A specific value' (checked). The 'A specific value' dropdown is set to 'Approved'. At the bottom are buttons for 'Save', 'Save & New', and 'Cancel'.

- Under Final Rejection Actions, Click on Add New and select Field Update
- Give the name as Update Status Rejected
- Unique name will be auto populated
- Select object as Claim
- Select Approval Status for Field to Update
- Select Picklist as field datatype
- Under specify new field value, Select a Specific value and select Rejected
- Click on Save

This screenshot shows the same Salesforce 'Field Updates' configuration page, but for a new entry named 'Update Status Rejected'. The 'Identification' section fields are: Name (Update Status Rejected), Unique Name (Update_Status_Rejected), Description (empty), Object (Claim), Field to Update (Approval Status), Field Data Type (Picklist), and the 'Re-evaluate Workflow Rules after Field Change' checkbox (unchecked). In the 'Specify New Field Value' section, the 'A specific value' radio button is selected, and the dropdown menu is set to 'Rejected'. The 'Save', 'Save & New', and 'Cancel' buttons are visible at the bottom.

- Activate the Approval Process.

3. Create a Record-Triggered Flow (After Save) that checks if the Claim Amount is \$>\$50,000\$ and submits the record to an Approval Process.

- Go to setup, In the quick find box, search for flows and click on flows.
- Click on New Flow
- Select Record Triggered Flow
- Select object as Claim
- Configure Trigger When a Record is created or updated
- Select All conditions are true for conditional requirements
- The conditions include:
 - Claim Amount Greater than 50000
 - Approval Status Equals New
- Select Every time a record is updated and meets the conditional requirements for When to run the flow for Updated Records
- Optimize the flow for Action and Related Records

The screenshot displays the Salesforce Flow Builder interface for a flow named "Submission Automation Flow - V1". The flow is configured as a Record-Triggered Flow. The configuration panel on the right shows the following settings:

- Select Object:** Claim
- Configure Trigger:** Trigger the Flow When: ☒ A record is created or updated
- Set Entry Conditions:** (Empty)

The flow canvas on the left shows the flow structure:

- Record-Triggered Flow Start** (Trigger)
- Run Immediately** (Start Action)
- Submit Claim to Approval** (Action)
- End** (End Action)

The flow is currently in the "Active" state, as indicated by the "Active" button in the top right corner of the configuration panel.

Flow Builder | Submission Automation Flow - V1 | Enable flow scrollability

Last saved on 12/2/2025, 03:44 PM | Active | Run | Debug | View Tests | Save As New Version | Save | Deactivate

Record-Triggered Flow
Start
Object: Claim
Trigger: A record is created or updated
Conditions: 2
Optimize for: Actions and Related Records
+ Add Scheduled Paths (Optional)
Open Flow Trigger Explorer for Claim

Run Immediately

Submit Claim to Approval
Action

End

Configure Start

Field: Claim Amount Operator: Greater Than Value: 50000

Field: Approval Status Operator: Equals Value: New

+ Add Condition

When to Run the Flow for Updated Records

☒ Every time a record is updated and meets the condition requirements
☐ Only when a record is updated to meet the condition requirements

Optimize Flow

Optimize the Flow for:

Fast Field Updates
Update fields on the record that triggers the flow to run. This high-performance flow runs **before the record is saved** to the database.

Actions and Related Records
Update any record and perform actions, like send an email. This more flexible flow runs **after the record is saved** to the database.

- Add an action element after the Start Element
- Label it as Submit Claim to Approval
- Under Set Input Values, For Record Id, Select Triggering Claim → Record ID
- For select Approval Process Name or Id, Enter the Approval Process Name(High Value Claim Approval)

Flow Builder | Submission Automation Flow - V1 | Enable flow scrollability

Last saved on 12/2/2025, 03:44 PM | Active | Run | Debug | View Tests | Save As New Version | Save | Deactivate

Record-Triggered Flow
Start
Object: Claim
Trigger: A record is created or updated
Conditions: 2
Optimize for: Actions and Related Records
+ Add Scheduled Paths (Optional)
Open Flow Trigger Explorer for Claim

Run Immediately

Submit Claim to Approval
Action

End

Submit for Approval

Submit Claim to Approval

* API Name
Submit_Claim_to_Approval

Description

Use values from earlier in the flow to set the inputs for the "Submit for Approval" core action. To use its outputs later in the flow, store them in variables.

Set Input Values

A₃ * Record ID
Triggering Claim__c > Record ID

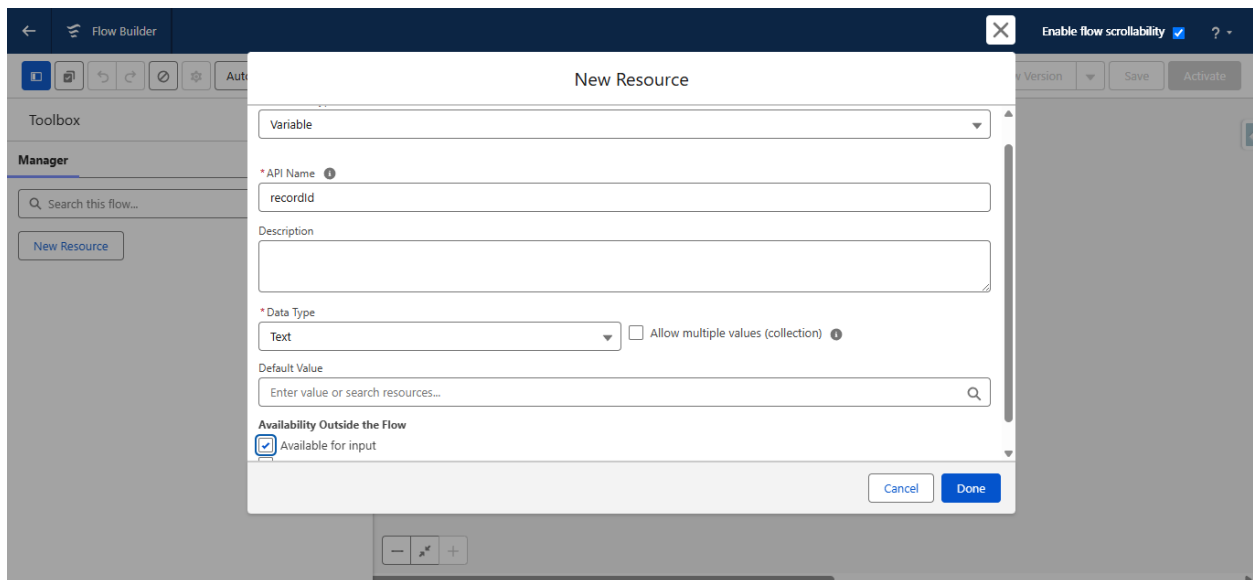
A₃ Approval Process Name Or ID
High Value Claim Approval

Included

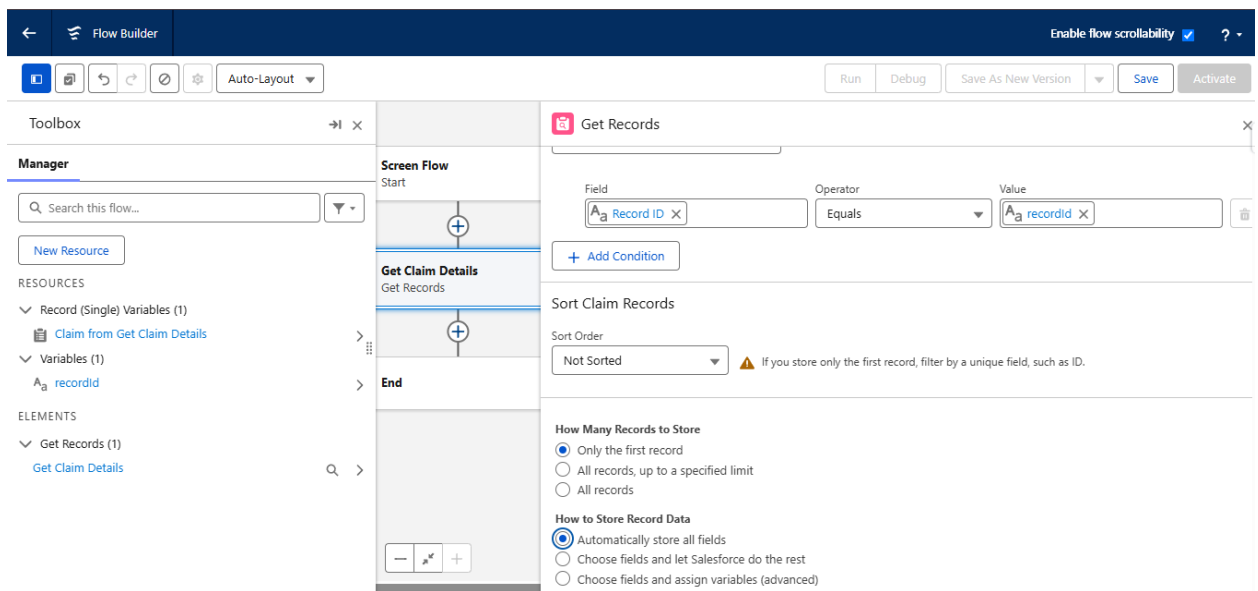
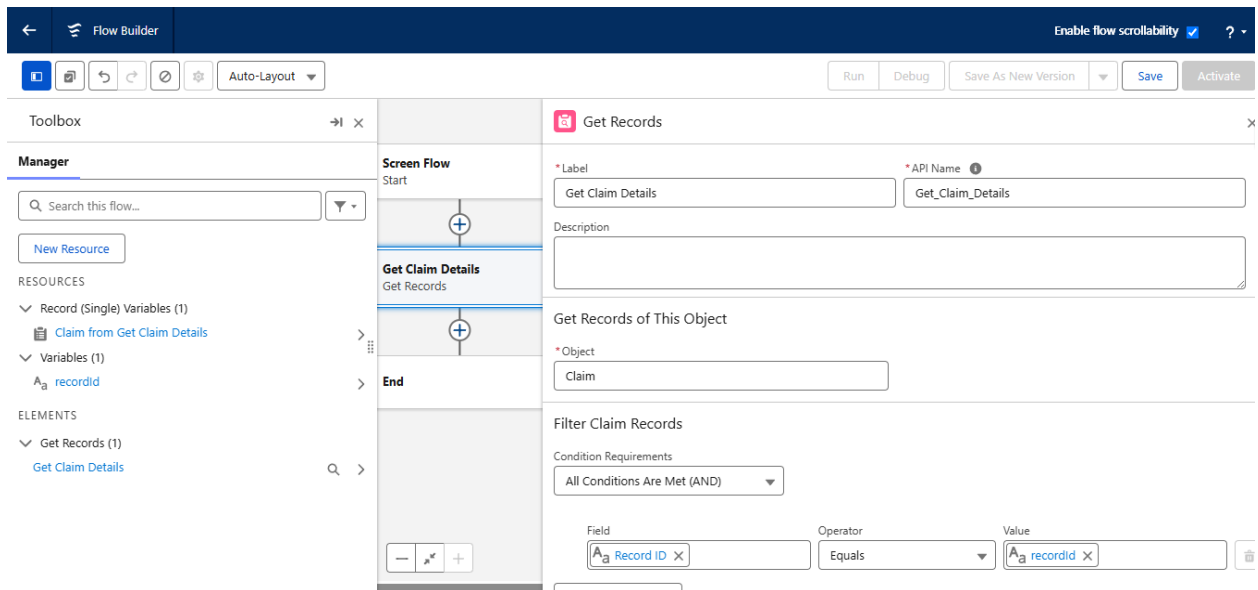
- Save the Flow with Submission Automation Flow and Activate the flow

4. Implement a Screen Flow for the approvers to quickly review details and approve/reject directly from the Claim record.

- Go to setup, Search for flows in the quick find box and click on flows.
- Click on New Flow, Choose Screen Flow
- Create a New Resource by clicking on New Resource button in the toolbox manager
- Resource Type would be Variable
- API Name would be recordId
- Data type would be Text
- Under Availability outside the flow, Check Available for Input
- Click on Done

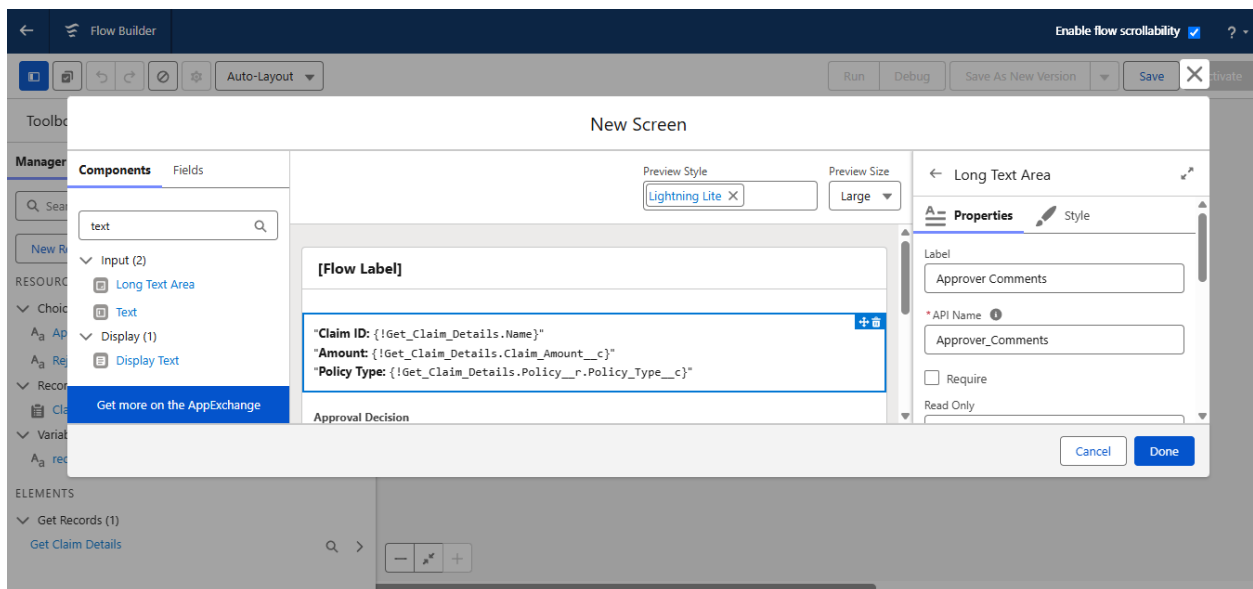


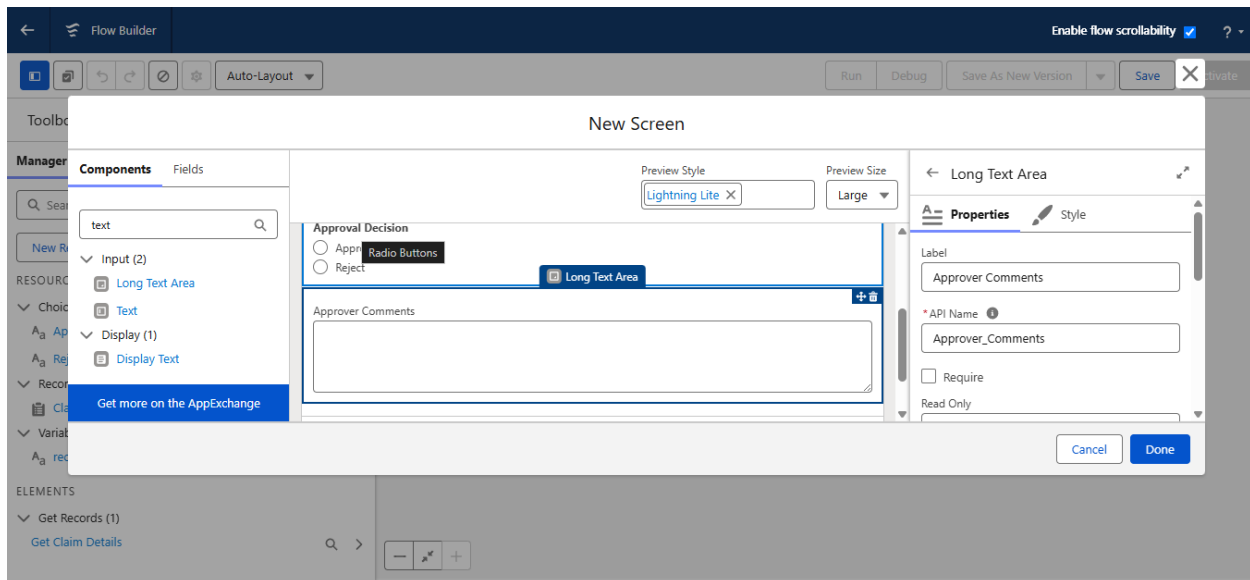
- Add the Get Records Element below the screen
- Label it as Get Claim Details
- Select object as Claim
- Select All conditions are required for conditional Requirements
- The Condition include:
- RecordID equals variable recordId
- Store only the first record and
- Automatically store all fields



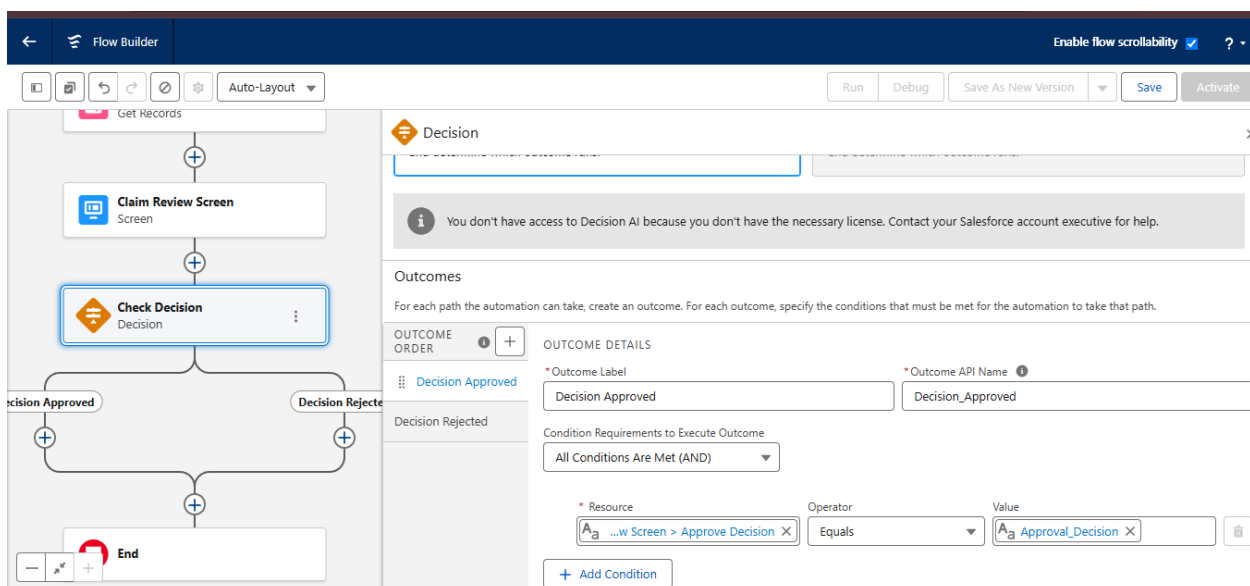
- Add a screen Element under the get Record element
- Label it as Claim Review Screen
- Add Display Text component to the flow canvas, and paste the following:
 - **Claim ID:** {!Get_Claim_Details.Name}"
 - **Amount:** {!Get_Claim_Details.Claim_Amount__c}"
 - **Policy Type:** {!Get_Claim_Details.Policy__r.RecordType.Name}"
- Add the Radio buttons component, Label it as Approval Decision
- API Name would be auto populated
- For Choice, Create a resource, Resource type would be choice

- Give the choice label as Approve
- Select Datatype as Text
- Enter the choice value as Approve
- Enter the input label as Approve
- Click on Done
- Add one more Choice, Create a resource, Resource type would be choice
- Give the choice label as Reject
- Select Datatype as Text
- Enter the choice value as Reject
- Enter the input label as Reject
- Click on Done
- Add Long Text area into the flow canvas and Label it as Approver Comments
- API Name would be Auto Populated
- Click on Done

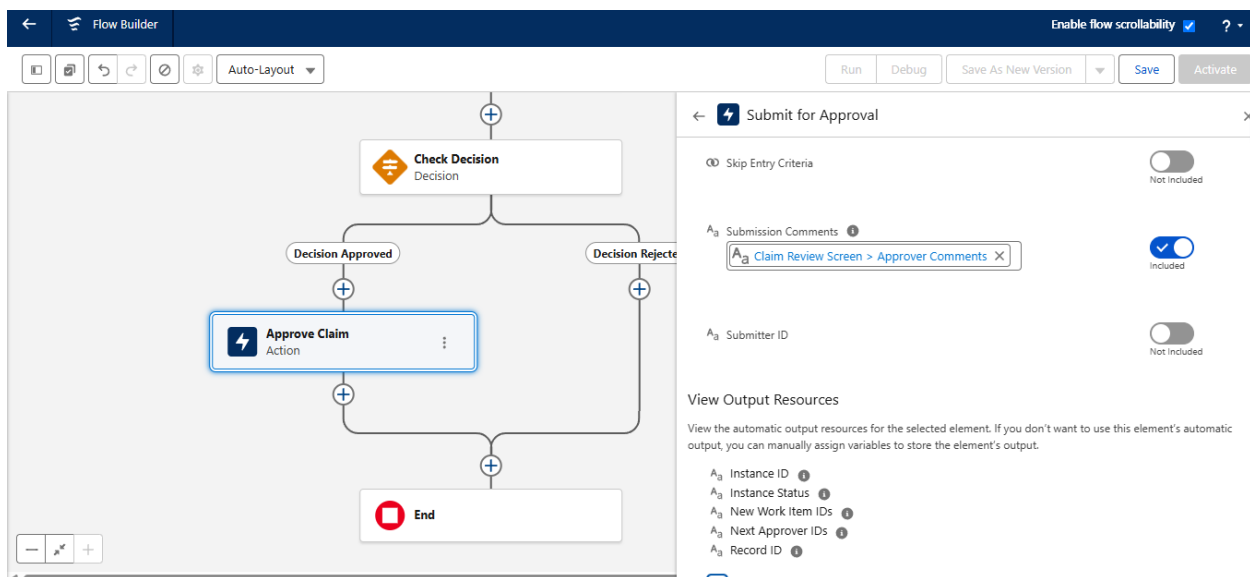
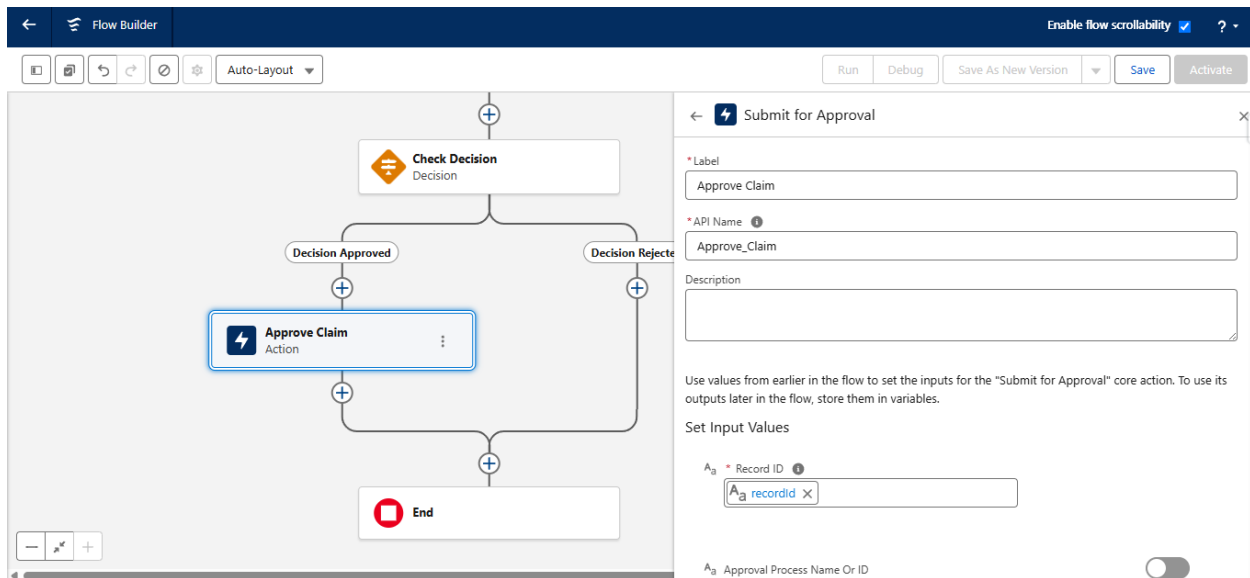




- Add Decision Element under the screen
- Label it as Check Decision
- Select Manually Select Conditions
- Set Outcome1:
 - Label it as Decision Approved
 - Select all conditions are met for conditional requirements
 - The condition includes:
 - Approve Decision From Claim Review Screen → Equals Approval Decision
- Set default Outcome:
 - Label it as Decision Rejected

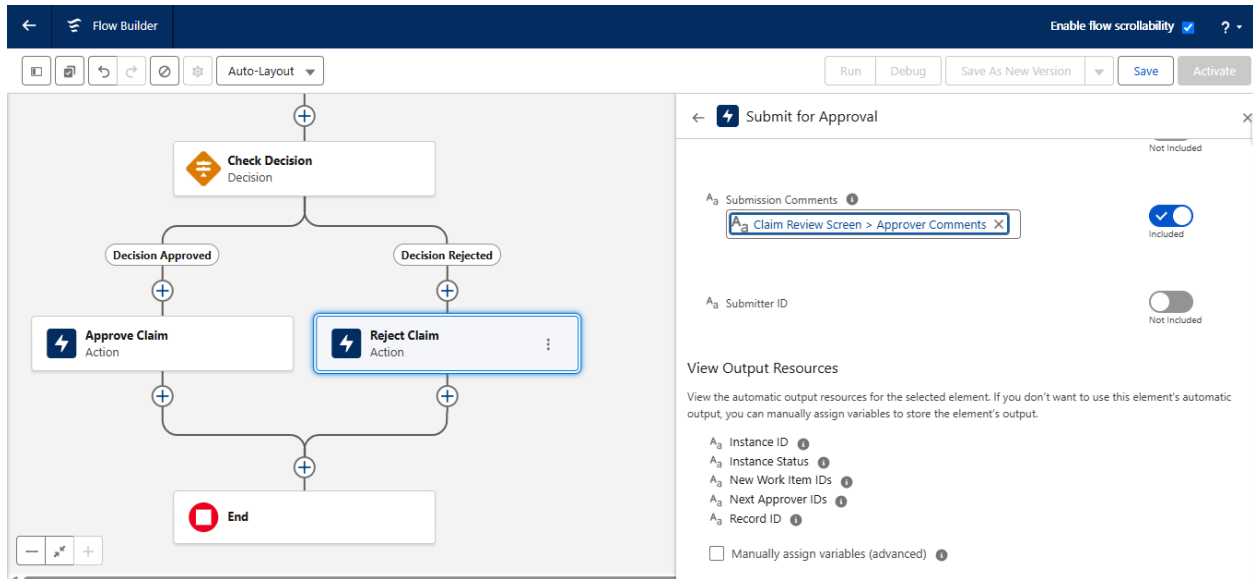
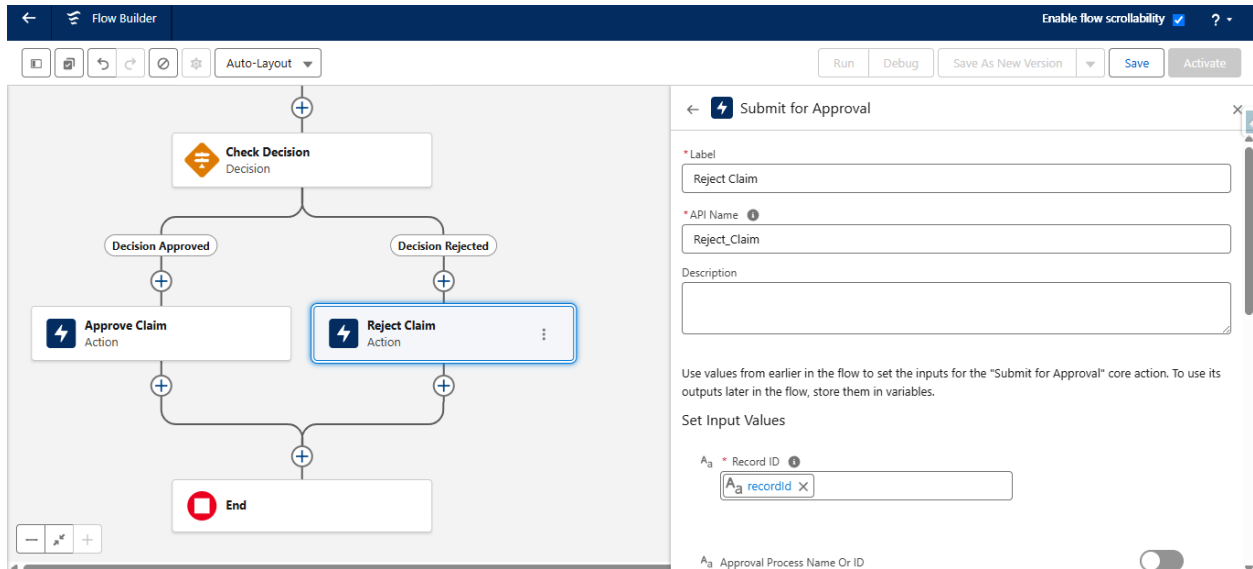


- Add Action Element Under Decision Approved Path
- Search for the submit for approval action and choose it
- Label it as Approve Claim
- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen



- Add Action Element Under Decision Rejected Path
- Search for the submit for approval action and choose it
- Label it as Reject Claim

- Under Set Input Values
- Select variable recordID for Record ID
- Include the submission comments and Enter Approver Comments from Claim Review Screen



- Save and Activate the flow with Name Claim Approver Screen Flow

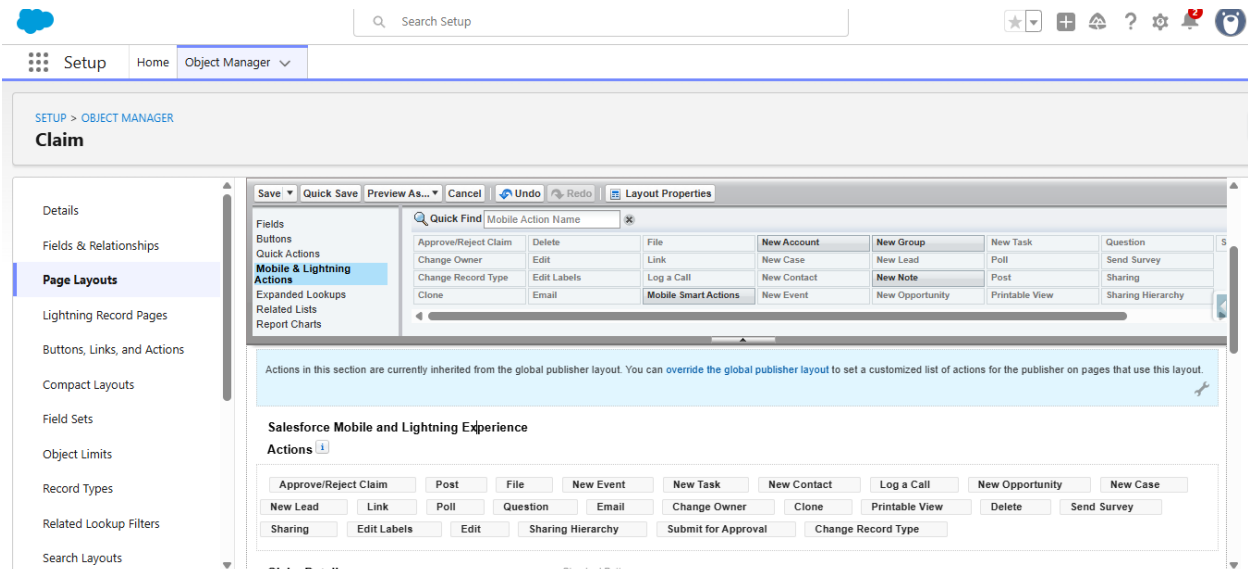
4.1 Create a Quick Action

- Go to setup, Click on Object Manager, Search for Claim Object and Select it
- In the left side panel, Click on Buttons, Links and Actions
- Click on Create New Action
- Select Action Type as Flow
- Select the flow that you have just created
- Label it as Approve/Reject Claim
- Name would be auto populated
- Click on Save

The screenshot shows the Salesforce Setup interface. At the top, there's a search bar and navigation tabs for Setup, Home, and Object Manager. The Object Manager is selected, and the 'Claim' object is chosen. On the left sidebar, 'Buttons, Links, and Actions' is highlighted. The main area shows the 'Approve/Reject Claim' action detail. The action is labeled 'Approve/Reject Claim', has the name 'Approve_Reject_Claim', and is associated with the 'Claim Approver Screen Flow'. It was created by 'Insurance Portal' on 12/5/2025 at 2:48 AM.

Action Detail		Object Name	
Label	Approve/Reject Claim	Object Name	Claim
Standard Label Type	Approve_Reject_Claim	Action Type	Flow
Name	Approve_Reject_Claim	Icon	⚡
Description	Claim Approver Screen Flow		
Flow	Claim Approver Screen Flow		
Created By	Insurance Portal, 12/5/2025, 2:48 AM	Modified By	Insurance Portal, 12/5/2025, 2:48 AM

- Add the quick action to the claim page layout
- In the left panel, Click on page layouts and open the claim page layout
- Go to mobile & lightning actions, Find the Approve/Reject Claim and Drag it to the Salesforce Mobile and Lightning Experience Actions
- Click on Save



5. Create an Apex test class for ClaimsAdjusterController with strong code coverage, including tests for claim retrieval and filtering logic.

- Click on the setup icon, Right click on Developer Console and click on open link in new tab.
- On the top left corner, click on file, click on new, click on Apex class and give it a name ClaimsAdjusterControllerTest
- Paste the following code:

```
@isTest
```

```
private class ClaimsAdjusterControllerTest {
```

```
    @testSetup
```

```
    static void setupTestData() {
```

```
        // Create a real active test user
```

```
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
```

```
User u = new User(
```

```
    FirstName = 'Test',
```

```
    LastName = 'User',
```

```
    Alias = 'tuser',
```

```
    Username = 'testuser1234343@example.com.test',
```

```
    Email = 'testuser123@example.com',
```

```
    TimeZoneSidKey = 'America/Los_Angeles',
```

```
    LocaleSidKey = 'en_US',
```

```
    EmailEncodingKey = 'UTF-8',
```

```
    LanguageLocaleKey = 'en_US',
```

```
    ProfileId = p.Id
```

```
);
```

```
insert u;
```

```
// Query Policy__c RecordType
```

```
Id policyRT =
```

```
Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName()
```

```
.get('Auto')
```

```
.getRecordTypeId();
```

```
Id claimRT =  
Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
```

```
.get('Accident')
```

```
.getRecordTypeId();
```

```
// Create Contact owned by same user
```

```
Contact con = new Contact(
```

```
    FirstName = 'John',
```

```
    LastName = 'Doe',
```

```
    OwnerId = u.Id
```

```
);
```

```
insert con;
```

```
// Create Policy owned by same user
```

```
Policy__c policy = new Policy__c(
```

```
    Customer__c = con.Id,
```

```
    RecordTypeId = policyRT,
```

```
OwnerId = u.Id,
```

```
VIN__c = '1A903284375893532'
```

```
);
```

```
insert policy;
```

```
// Create Claim owned by same user
```

```
Claim__c claim = new Claim__c(
```

```
Policy__c = policy.Id,
```

```
RecordTypeId = claimRT,
```

```
Claim_Amount__c = 5000,
```

```
Date_of_Loss__c = Date.today().addDays(-10),
```

```
OwnerId = u.Id
```

```
);
```

```
insert claim;
```

```
System.debug('Claim inserted: '+claim.Id);
```

```
}
```

```
@isTest
```

```
static void testGetAssignedClaims() {
```

```
    User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
```

```
    System.runAs(u) {
```

```
        Test.startTest();
```

```
        List<ClaimsAdjusterController.ClaimWrapper> results =
```

```
            ClaimsAdjusterController.getAssignedClaims();
```

```
        System.debug('Result is: '+results);
```

```
        Test.stopTest();
```

```
        //System.assert(results.size() > 0,
```

```
            //'Expected at least one wrapper returned');
```

```
        ClaimsAdjusterController.ClaimWrapper wrap = results[0];
```

```
        System.assertNotEquals(null, wrap.claimId);
```

```
        System.assertNotEquals(null, wrap.claimNumber);
```

```
        System.assertEquals('New', wrap.status);
```

```
        System.assertEquals(5000, wrap.claimAmount);
```

```
System.assert(wrap.policyHolderName.contains('John'));
```

```
System.assertNotEquals('Unknown', wrap.policyType);
```

```
}
```

```
}
```

```
}
```

- Save the file using CTRL+S
- On the top right corner, you will find Run Test, Click on it to run the test class
- To check the test results, Down go for tests and you can see the how much percentage it covers your apex class

The screenshot displays the Salesforce IDE interface. The top pane shows the Apex test class `ClaimsAdjusterControllerTest.apxc` with the following code:

```
1 @isTest
2 private class ClaimsAdjusterControllerTest {
3
4     @testSetup
5     static void setupTestData() {
6
7         // Create a real active test user
8         Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
9         User u = new User(
10             FirstName = 'Test',
11             LastName = 'User',
12             Alias = 'tuser',
13             Username = 'testuser123434@example.com.test',
14             Email = 'testuser123@example.com',
```

The bottom pane shows the **Tests** tab with a list of test runs. The most recent successful run is highlighted:

Status	Test Run	Enqueued Time	Duration	Failures	Total
✗	TestRun @ 11:58:17 am			1	1
✗	TestRun @ 12:01:35 pm			1	1
✗	TestRun @ 12:07:16 pm			1	1
✗	TestRun @ 12:14:03 pm			1	1
✗	TestRun @ 12:22:36 pm			1	1
✗	TestRun @ 12:27:50 pm			1	1
✓	TestRun @ 12:29:39 pm			0	1

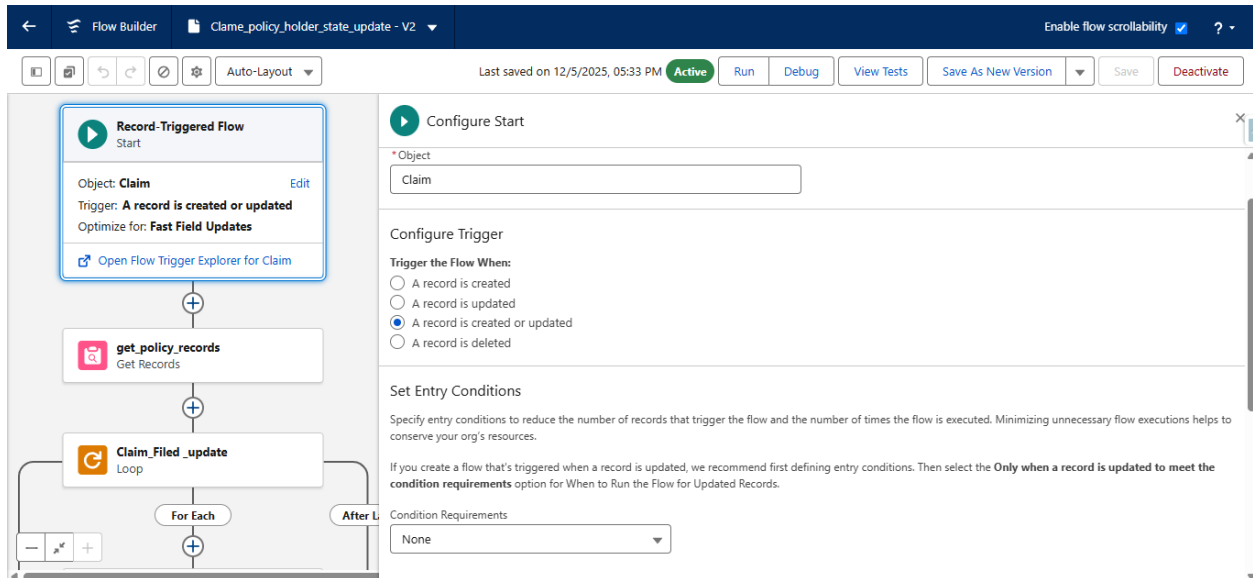
On the right side of the Tests tab, the **Overall Code Coverage** summary is shown:

Class	Percent	Lines
Overall	48%	
ClaimsAdjusterController	86%	20/23
PremiumCalculator	0%	0/18

6. Configure sharing logic so adjusters can work with claims associated with their assigned territory or state.

- Go to setup, Search for flows in the quick find box and click on flows
- Click on New Flow
- Select Record Triggered Flow

- Select the object as claim
- Configure trigger when a record is created or updated
- Set entry conditions as none
- Optimize the flow for Updates



- Add get Records element into the flow canvas
- Label it as get_policy_records
- API name will be auto populated
- Set filter as None
- Store all the records
- Select Choose fields and let salesforce do the rest for How to store Record Data
- Under Select Policy fields to store fields in Variable, Select Policy State Field

Flow Builder interface showing the configuration of a Record-Triggered Flow named "Clame_policy_holder_state_update - V2". The flow is triggered by "A record is created or updated" for the object "Claim". The flow steps are:

- Record-Triggered Flow Start** (Object: Claim, Trigger: A record is created or updated, Optimize for: Fast Field Updates)
- get_policy_records** (Get Records)
- Claim_Filed_update** (Loop)
- For Each** (Iterating over collection)
- After Last**

The right-hand pane shows the configuration for the **Get Records** element:

- Label:** get_policy_records
- API Name:** get_policy_records
- Object:** Policy
- Filter Policy Records:** With no conditions, the flow retrieves all Policy records. Condition Requirements: None—Get All Policy Records
- Sort Policy Records:** Not Sorted

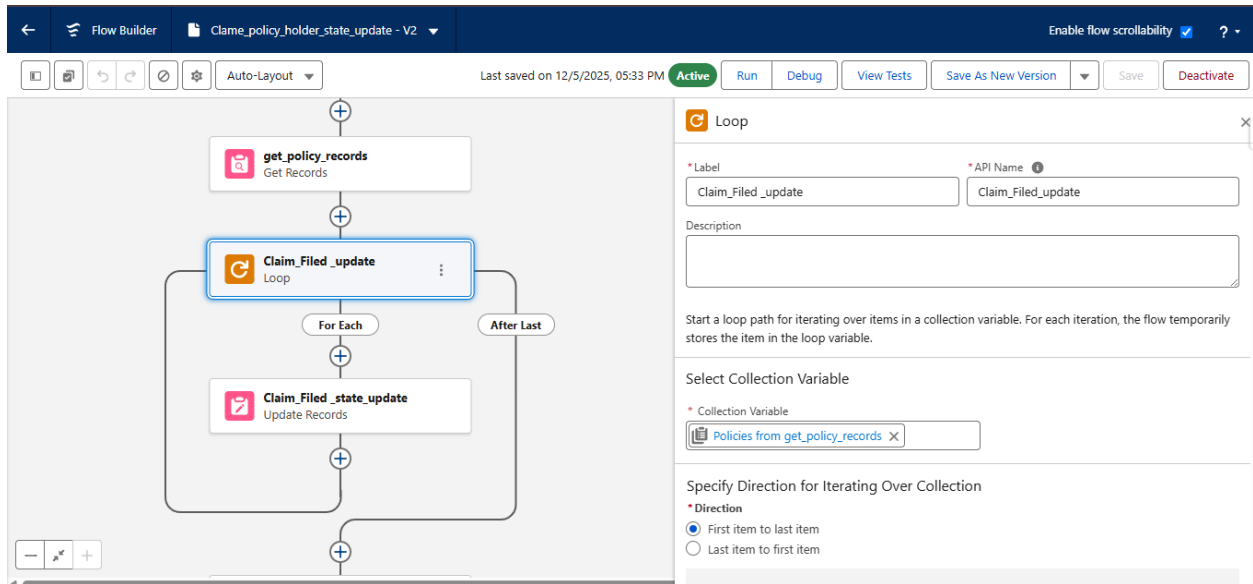
Flow Builder interface showing the configuration of the **Claim_Filed_update** Loop element. The flow steps are:

- Record-Triggered Flow Start** (Object: Claim, Trigger: A record is created or updated, Optimize for: Fast Field Updates)
- get_policy_records** (Get Records)
- Claim_Filed_update** (Loop)
- For Each** (Iterating over collection)
- After Last**

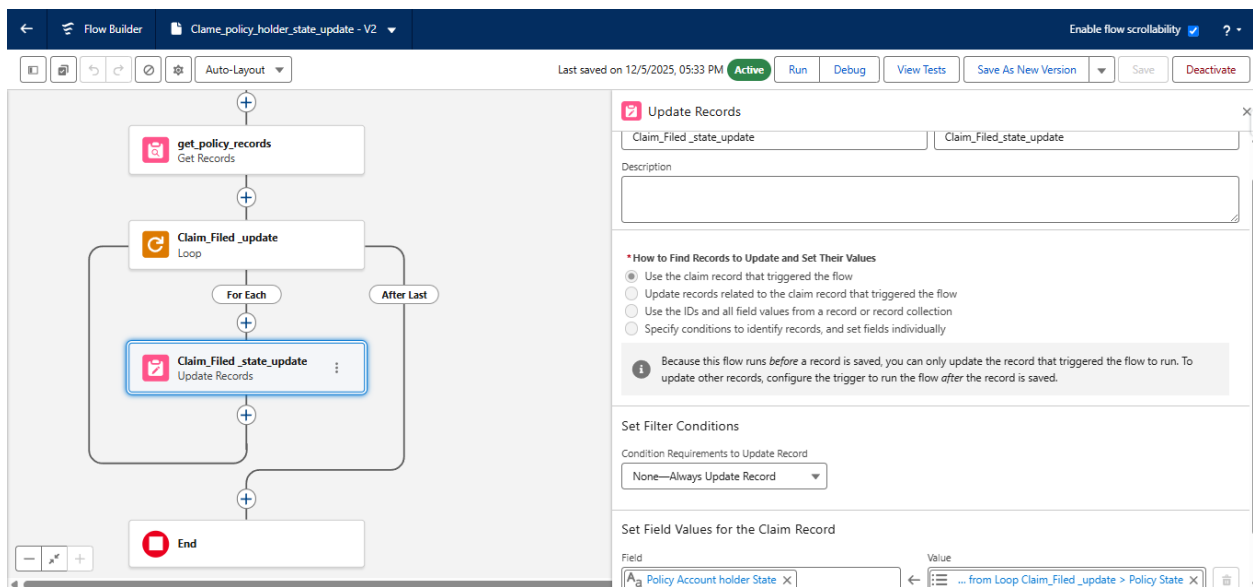
The right-hand pane shows the configuration for the **Get Records** element:

- Sort Policy Records:** Not Sorted
- How Many Records to Store:** All records, up to a specified limit
- How to Store Record Data:** Choose fields and let Salesforce do the rest
- Select Policy Fields to Store in Variable:**
 - Field: Id
 - Field: Policy_State__c
 - + Add Field

- Add the Loop Element into the canvas after the get Records element
- Label it as Claim_Filed_update
- API name will be auto populated
- Under Select Collection Variables, Select policies from Get Policies from Get Policy Records
- Under Specify direction for Iterating over collection, Select first item to last item



- Under For each, Add the update records element into the canvas
- Label it as Claim_Filed_state_update
- API name will be auto populated
- Set filter conditions to NONE
- Under Set field values for the claim record, Select policy state from current loop and set it to the Policy Account Holder State field



- Save and Activate the flow with name Claim Policy Holder State Update

6.1 Define Sharing Rules for Claim Object

- Go to Set up, In the quick find box, Search for Sharing Settings
- Click on Sharing settings
- Under Manage Sharing Settings for, Select Claim Object
- Scroll down to find Claim Sharing Rules
- Click on new
- Label it as Share Records based on Policy State CA
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals CA
- Give Access Level as Read/Write
- Click on Save

Setup Home Object Manager

Search Setup

Q sharing

Security

Guest User Sharing Rule Access Report

Sharing Settings

Didn't find what you're looking for? Try using Global Search.

SETUP
Sharing Settings

You can use sharing rules only to grant wider access to data, not to restrict access.

Label: Share Records based on Policy State CA
Rule Name: Share_Records_based_on_Policy_State_CA
Description:

Step 1: Select your rule type

Criteria	Field	Operator	Value	
	Policy Account holder State	equals	CA	AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		

[Add Filter Logic...](#)

Additional Options

☒ Include records owned by users who can't have an assigned role

Share with: Group: Claim Adjusters - California

Access Level: Read/Write

Created By: Insurance Portal, 12/7/2025, 8:34 PM

Modified By: Insurance Portal, 12/7/2025, 8:34 PM

- Again Click on New
- Label it as Share Records based on Policy State
- Rule Name will be Auto Populated
- Set the criteria to Policy Account Holder State Equals TX
- Give Access Level as Read/Write
- Click on Save

Search Setup

Setup Home Object Manager

Q sharing

Security

Guest User **Sharing** Rule Access Report

Sharing Settings

Didn't find what you're looking for? Try using Global Search.

SETUP
Sharing Settings

Label: Share Records based on Pol

Rule Name: Share_Records_based_on_f

Description:

Step 1: Select your rule type

Criteria	Field	Operator	Value	
	Policy Account holder State	equals	TX	AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		AND
	--None--	--None--		AND

[Add Filter Logic...](#)

Additional Options: ☐ Include records owned by users who can't have an assigned role

Share with: Group: Claim Adjusters- Texas

Access Level: Read/Write

Modified By: Insurance Portal, 12/7/2025, 8:33 PM

Created By: Insurance Portal, 12/7/2025, 8:33 PM

7. Finalize role-based Permission Sets so Agents focus on quoting, Adjusters manage claims, and Managers handle reporting and approvals.

- Go to setup, In the quick find box, Search for Permission Sets, Click on Permission Sets
- Click on New
- Give the Label as Insurance Agent Access
- API Name will be Auto Populated

Search Setup

Setup Home Object Manager

Q permisio

Users

Permission Set Groups

Permission Sets

Custom Code

Custom Permissions

Didn't find what you're looking for? Try using Global Search.

SETUP
Permission Sets

Save Cancel

Enter permission set information

Label: Insurance Agent Access

API Name: Insurance_Agent_Access

Description:

Session Activation Required: ☐

Select the type of users who will use this permission set

Who will use this permission set?

-Choose "--None--" if you plan to assign this permission set to multiple users with different user and permission set licenses.

-Choose a specific user license if you want users with only one license type to use this permission set.

-Choose a specific permission set license if you want this permission set license auto-assigned with the permission set.

Not sure what a permission set license is? [Learn more here.](#)

License: --None--

Save Cancel

- Give the access for Objects
- Give Create, Read for Policy object
- Give Read, Edit to contact Object
- Give Read only to Claim object
- Make Flow User Enabled under System permissions
- Make PremiumCalculator Apex class enabled

7.1 Create Claims Manager Access Permission set

- Click on New
- Give the Label as Claims Manager Access
- API Name will be Auto Populated

The screenshot shows the Salesforce Setup interface for creating a new permission set. The left sidebar shows the navigation menu with 'Setup' selected. The main content area is titled 'Permission Sets'. The 'Enter permission set information' section is active, showing the following fields:

- Label:** Claims Manager Access
- API Name:** Claims_Manager_Access
- Description:** (Empty)
- Session Activation Required:** ☐

Below this section, the 'Select the type of users who will use this permission set' section is visible, with a dropdown menu set to '--None--'.

- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to contact Object
- Give Read, Edit, Delete to Claim object
- Make Run Reports Enabled under System permissions
- Make Manage Approvals enabled under system permissions

7.2 Create Claims Adjuster Access Permission Set

- Click on New
- Give the Label as Claims Adjuster Access
- API Name will be Auto Populated

- Give the access to objects
- Give Read Only for Policy object
- Give Read, Edit to Claim object
- Make ClaimsAdjusterController Enabled under Apex Classes

6. Project Outcome

The implemented Salesforce solution brings policy configuration, guided quotation, claim processing, approval, and adjuster access into a single application. The design supports multiple insurance lines while combining declarative automation with Apex and Lightning Web Components.

4. Supports Auto, Property, and Life policy data through Record Types, Field Sets, and validation rules.
5. Automates premium calculation, claim routing, and high-value claim approval using Apex and Salesforce Flows.
6. Provides an LWC-based claims dashboard together with sharing rules and Permission Sets for role-based access.